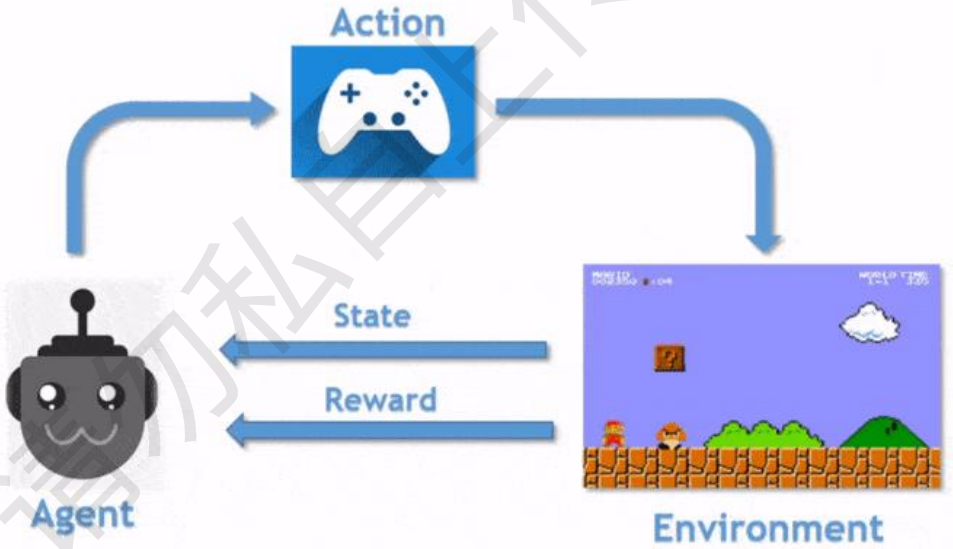
Agent与深度强化学习

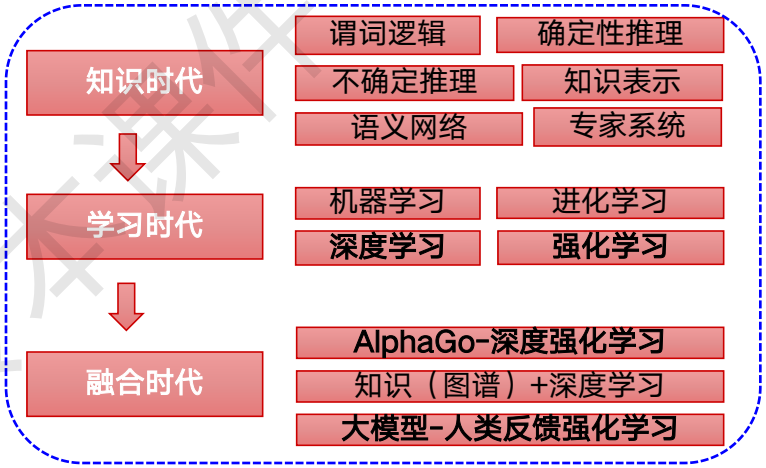


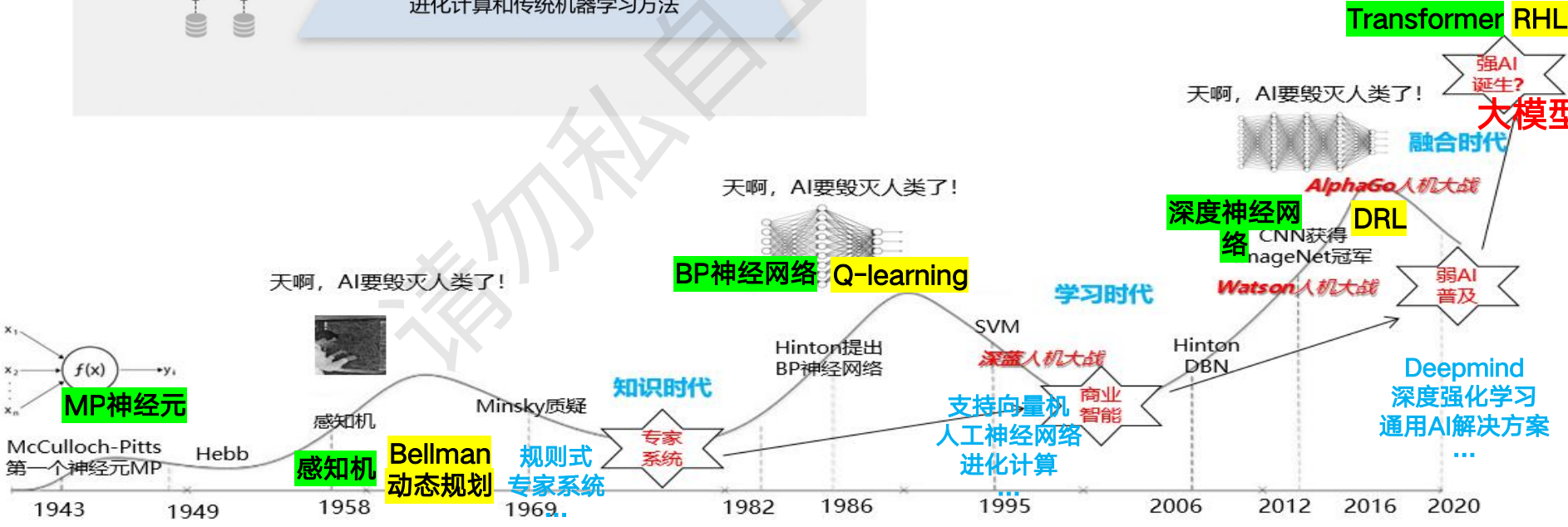
计算机与人工智能学院
马冲

请勿私自上传本课件

AI近70年发展的技术主线：神经网络与强化学习







请勿私自上传本课件

Agent与深度强化学习

1.1 什么是Agent

1.2 Agent结构设计

1.3 强化学习概述

1.4 深度强化学习

1.5 应用案例分析

请勿私自上传本课件

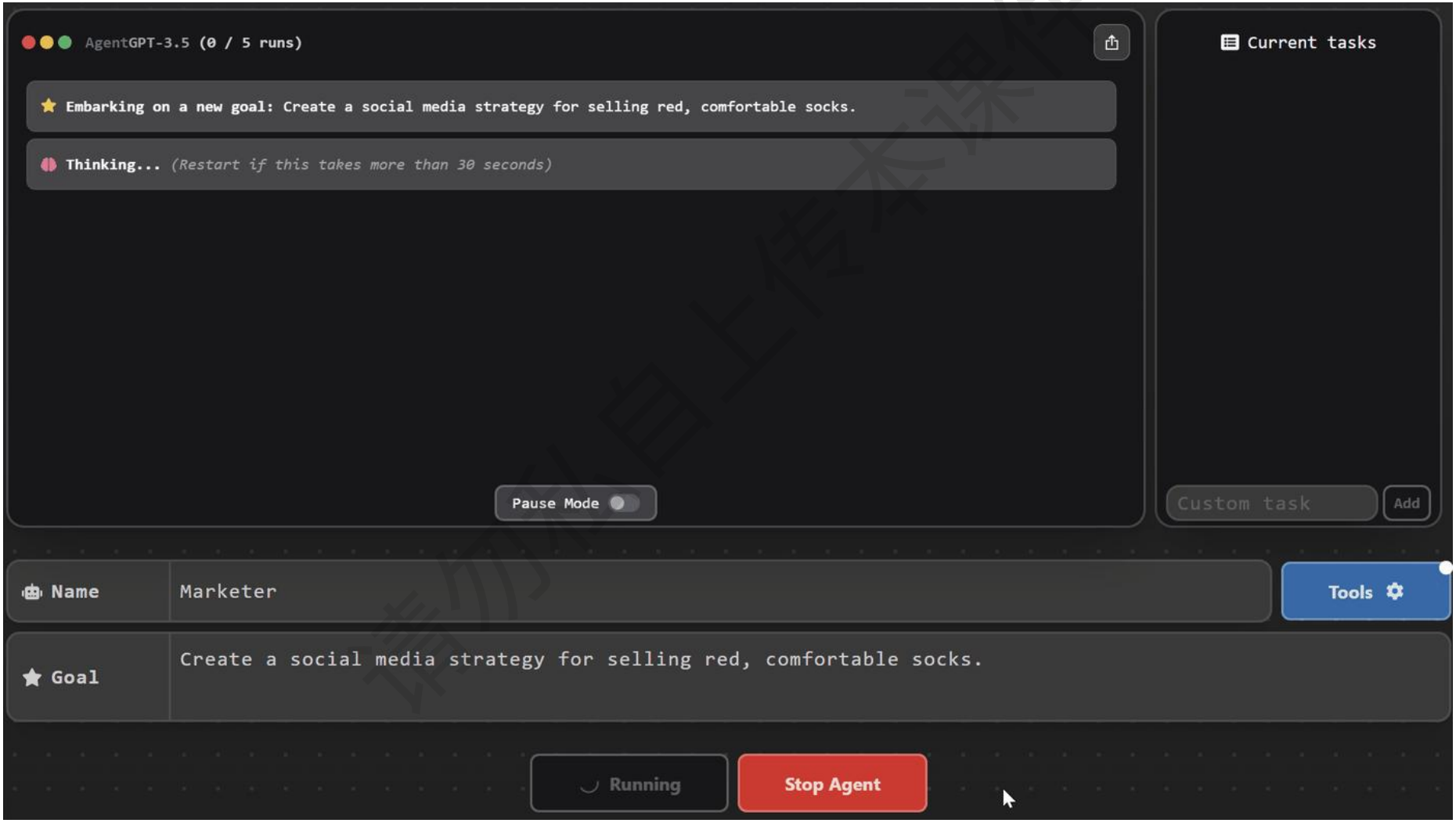
1.1 什么是Agent

- 为什么需要引入智能体？



1.1 什么是Agent

- 为什么需要引入智能体？



1.1 什么是Agent

- **为什么需要引入智能体？**

  - 我们，作为一种智能体，为什么能够思考？大脑为什么能够感知、理解、预测和应对一个远比自身庞大和复杂的世界？---**理解人类，理解智能体**
  - 人工智能要走得更远：不仅试图理解智能体，而且要**建造智能体**---制造出像人类一样完成某些智能任务的系统(软件)。

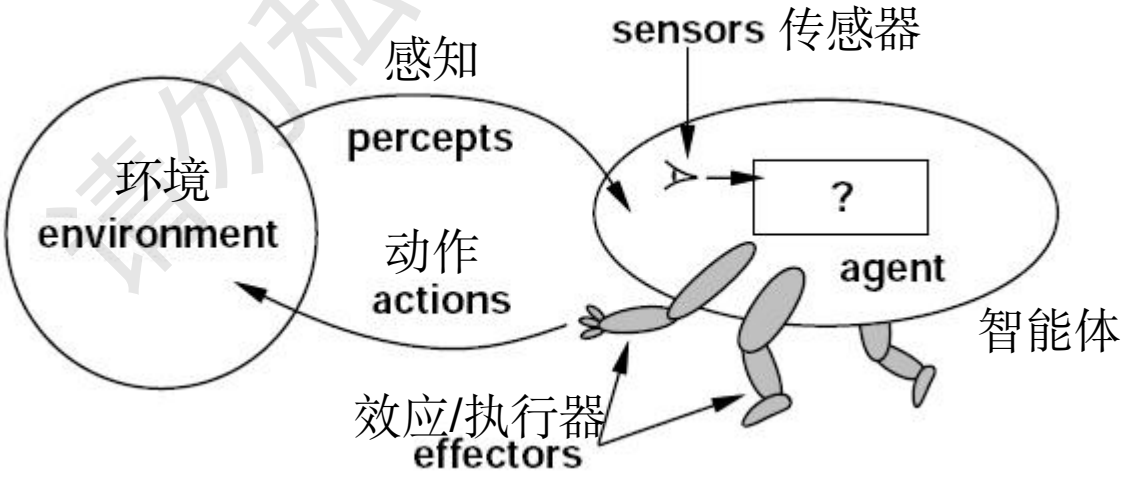


1.1 什么是Agent

智能体基本概念

智能体（Agent）：通过传感器感知所处环境并通过执行器对该环境产生作用的计算机程序及其控制的硬件

- ✓ **感知信息**：表示任意给定时刻智能体的感知输入 / **感知序列**：该智能体所收到的所有输入数据的完整历史
- ✓ **智能体函数**：把任意给定感知序列映射到智能体行动的描述 **智能体程序**：抽象的智能体函数的一个具体行动步骤的实现，该程序在智能体自身结构上运行

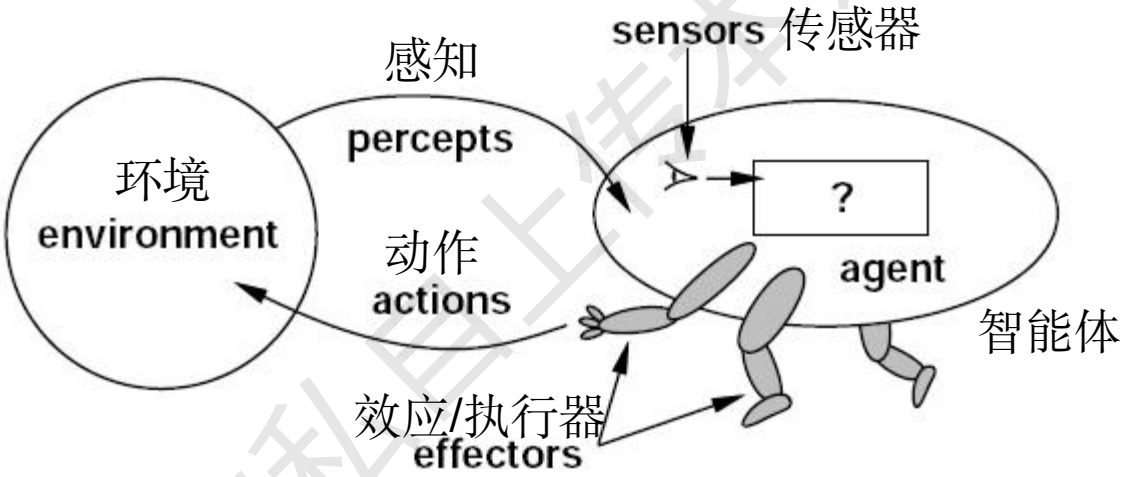


请勿私自上传课件

1.1 什么是Agent

智能体基本概念

Agent可以看做是一个自动执行的实体，通过传感器感知环境，通过效应器作用于环境。



- 构建Agent的任务就是设计Agent程序，即**实现Agent从感知到动作的映射**。智能体程序要在某个具备实际传感器和执行器的计算装置上运行，该装置称为体系结构。智能体程序具有这样的框架：从传感器得到当前感知信息作为输入，返回一个行动交给执行器

智能体 = 体系结构 + 程序

1.1 什么是Agent

- 智能体与任务环境

设计智能体需综合考虑:

PEAS (Performance, Environment, Actuators, Sensors) 性能/环境/执行器/传感器

智能家居Agent系统

性能度量	环境	执行器	传感器
- 用户满意度（语音指令响应准确率、任务完成率） - 响应延迟（<1秒） - 能源效率（自动关灯/调温节省电量） - 隐私保护合规性（不泄露用户数据）	- 家庭室内环境（客厅、卧室、厨房等） - 智能家居设备网络（灯光、空调、窗帘、音响、摄像头、门锁等） - 多用户共存（家庭成员语音/行为差异） - 网络连接状态（Wi-Fi/蓝牙稳定性）	- 控制智能设备的 API 接口（开/关灯、调节温度、播放音乐） - 语音合成模块（TTS，用于回答用户） - 手机 App 推送通知 - 显示屏内容更新（如智能中控面板）	- 麦克风（接收语音指令） - 摄像头（可选，用于人脸识别/手势控制） - 温湿度/光照/人体红外传感器 - 设备状态反馈（如〝空调已开启〞） - 用户日历/习惯数据（来自手机或云端）

请勿私自上传本课件

1.1 什么是Agent

- 智能体与任务环境

设计智能体需综合考虑:

PEAS (Performance, Environment, Actuators, Sensors) 性能/环境/执行器/传感器

多智能体交通信号控制系统

性能度量	环境	执行器	传感器
- 平均车辆等待时间最小化 - 路口通行效率（单位时间通过车辆数） - 排队长度均衡性 - 紧急车辆优先通行成功率 - 碳排放减少量	- 城市道路交叉口网络 - 实时车流动态（高峰/平峰） - 行人过街需求 - 天气/事故等突发事件 - 其他路口信号灯状态（多智能体协同环境）	- 交通信号灯控制器（红/黄/绿切换） - 可变信息板（VMS）显示提示 - 与车联网（V2I）平台通信接口 - 向导航 App 发送建议	- 地磁线圈/摄像头/雷达（检测车流量、排队长度） - 行人按钮/红外感应 - GPS 浮动车数据（来自网约车/出租车） - 紧急车辆 RF 信号识别

请勿私自上传本课件

1.1 什么是Agent

- **智能体与任务环境**

设计智能体需综合考虑:

PEAS (Performance, Environment, Actuators, Sensors) 性能/环境/执行器/传感器

辩论系统：双Agent对抗式观点生成与反驳

性能度量	环境	执行器	传感器
- 论点逻辑严密性（由人类评委或语言模型评分） - 反驳相关性与杀伤力 - 语言流畅性与说服力 - 观点新颖性（避免陈词滥调） - 辩论回合连贯性	- 辩题主题（如“AI应取代人类法官”） - 对手 Agent 的上一轮发言 - 知识库/事实数据库（用于支撑论点） - 辩论规则（时间限制、发言轮次）	- 自然语言生成模块（输出辩论语句） - 论点结构化输出（前提-结论格式） - 情感/语气控制（激昂、理性等风格）	- 对手发言文本输入（NLP 理解其论点、漏洞） - 辩题关键词与立场标签 - 实时事实核查 API（验证对方陈述真伪） - 用户/评委反馈信号（用于强化学习）

1.1 什么是Agent

- **智能体与任务环境**

设计智能体需综合考虑:

PEAS (Performance, Environment, Actuators, Sensors) 性能/环境/执行器/传感器

游戏AI自动设计系统

性能度量	环境	执行器	传感器
- AI 在游戏中的胜率/得分 - 策略多样性（避免同质化） - 泛化能力（在未见关卡/对手下表现） - 计算资源消耗（进化代数、种群大小）	- 游戏模拟器（如 StarCraft II、Atari、自定义 RTS） - 对手 AI 或人类玩家行为 - 游戏规则与状态空间 - 随机种子（用于公平评估）	- 输出 AI 控制策略（神经网络权重、决策树、行为树） - 向游戏引擎注入控制指令（移动、攻击、建造等） - 保存最优个体到模型库	- 游戏状态观测（屏幕像素、游戏内变量、地图信息） - 对手行为日志 - 本局胜负/得分反馈 - 多样性指标（如基因距离）

请勿私自上传本课件

1.1 什么是Agent

- **智能体与任务环境**

设计智能体需综合考虑:

PEAS (Performance, Environment, Actuators, Sensors) 性能/环境/执行器/传感器

城市级空气质量预测与可视化平台

性能度量	环境	执行器	传感器
- 预测准确率（PM 2.5 的MAE/RMSE） - 预警提前量（如提前 24 小时预警污染） - 空间分辨率（街区级 vs 城市级） - 系统响应速度（查询 < 2 秒） - 用户使用率/满意度	- 城市地理与气象条件（风速、湿度、温度） - 污染源分布（工厂、交通干道） - 历史空气质量数据 - 实时卫星/地面监测站数据流	- Web/API 输出预测结果 - 动态可视化地图（热力图、趋势动画） - 推送健康预警短信/App 通知 - 向环保部门提供决策报告	- 地面空气质量监测站（PM2.5、NO_2 、SO_2 等） - 卫星遥感数据（AOD 气溶胶光学厚度） - 气象站数据（风向、降水） - 社交媒体舆情（如“今天雾霾好重”） - 交通流量/工业活动数据（间接污染源）

课件仅供上传本课件

1.1 什么是Agent

- **智能体与任务环境**

任务环境的属性:

- ✓ 完全可观察vs部分可观察：获取环境的完整状态，一般难以做到
- ✓ 确定性的 vs 随机的：市内交通环境是随机的
- ✓ 片段式的 vs 延续式的：分拣零件机器人的决策只需建立在当前零件基础上，而下棋，驾驶，句法分析都是延续式的
- ✓ 离散的 vs 连续的：下棋是离散的，驾驶汽车是连续的
- ✓ 单智能体 vs 多智能体：城市交通控制和游戏AI对弈都是多智能体环境

请勿私自上传本课件

Agent与深度强化学习

1.1 什么是Agent

1.2 Agent结构设计

1.3 强化学习概述

1.4 深度强化学习

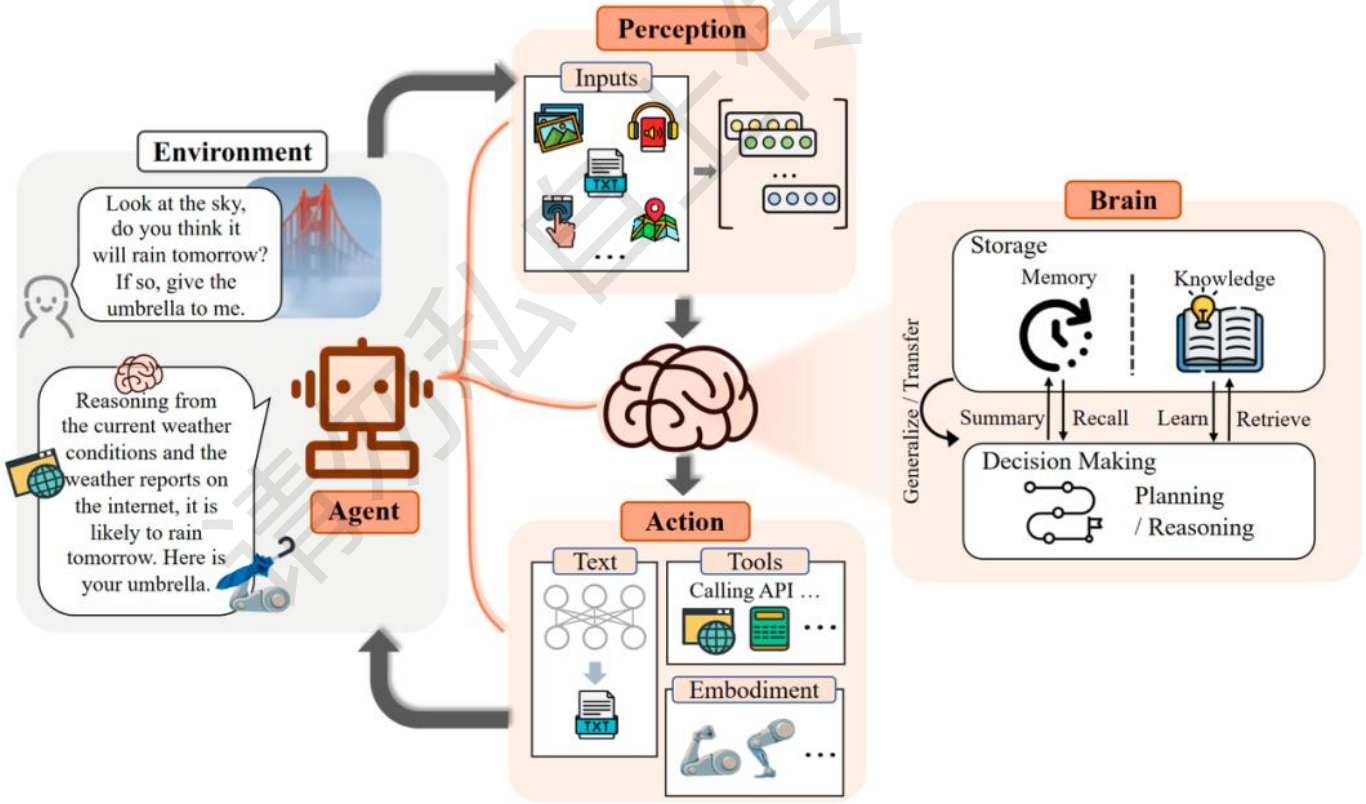
1.5 应用案例分析

请勿私自上传本课件

1.2 Agent结构设计

Agent结构设计要解决的关键问题

- ✓ Agent由哪些模块组成，模块之间如何交互信息
- ✓ Agent感知到的信息如何影响它的行为和内部状态
- ✓ 如何将这些模块用软硬件的方式组合成有机整体



1.2 Agent结构设计

Agent理论模型

Agent的理论模型研究主要从逻辑、行为、心理、社会等角度出发，对Agent的本质进行描述，为Agent系统创建奠定基础。

理性Agent模型(BDI模型)

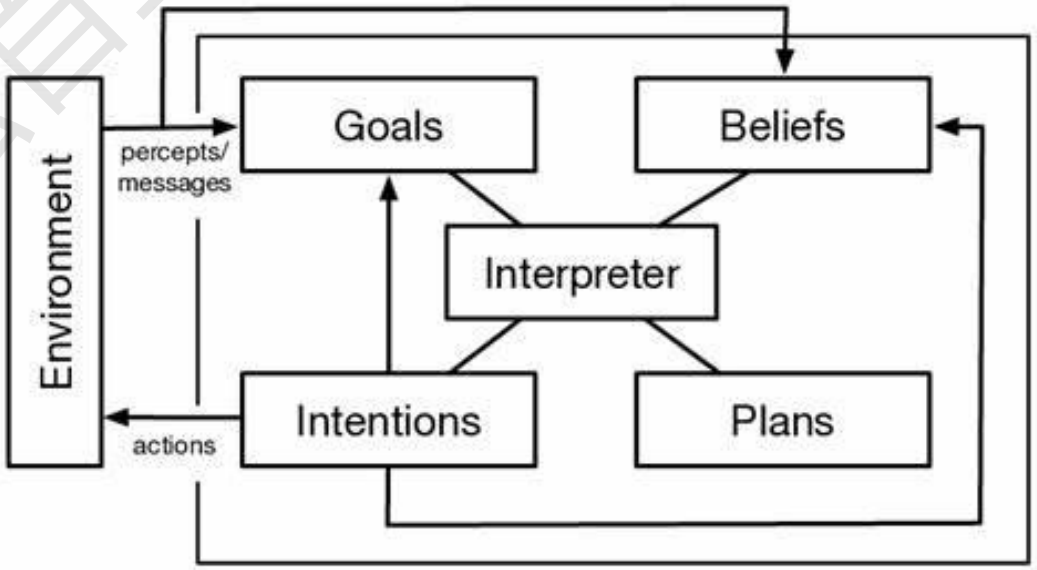
Belief——信念，Agent对环境的基本看法。——我知道什么

Desire——愿望，Agent想要实现的状态，即目标。——我想做什么

Intention——意图，目标的子集。——我决定做什么

BDI模型思想：

- Agent不只是‘能做’，更要‘懂’和‘想’；
- Agent有自己的‘世界观’（信念）、‘人生目标’（愿望）和‘行动计划’（意图）；
- 这种类人的心理结构让它能在复杂世界中做出合理、一致、可持续的决策。



1.2 Agent结构设计

Agent理论模型

Agent的理论模型研究主要从逻辑、行为、心理、社会等角度出发，对Agent的本质进行描述，为Agent系统创建奠定基础。

理性Agent模型(BDI模型)

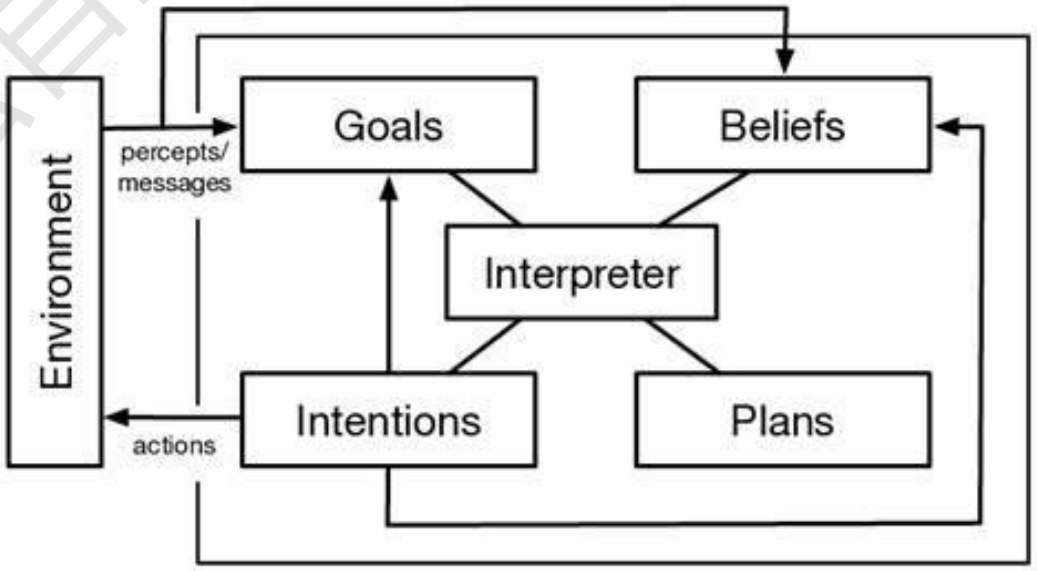
Belief——信念，Agent对环境的基本看法。——我知道什么

Desire——愿望，Agent想要实现的状态，即目标。——我想做什么

Intention——意图，目标的子集。——我决定做什么

BDI模型思想：

- Agent不只是‘能做’，更要‘懂’和‘想’；
- Agent有自己的‘世界观’（信念）、‘人生目标’（愿望）和‘行动计划’（意图）；
- 这种类人的心理结构让它能在复杂世界中做出合理、一致、可持续的决策。



1.2 Agent结构设计

- **理性Agent模型-BDI**

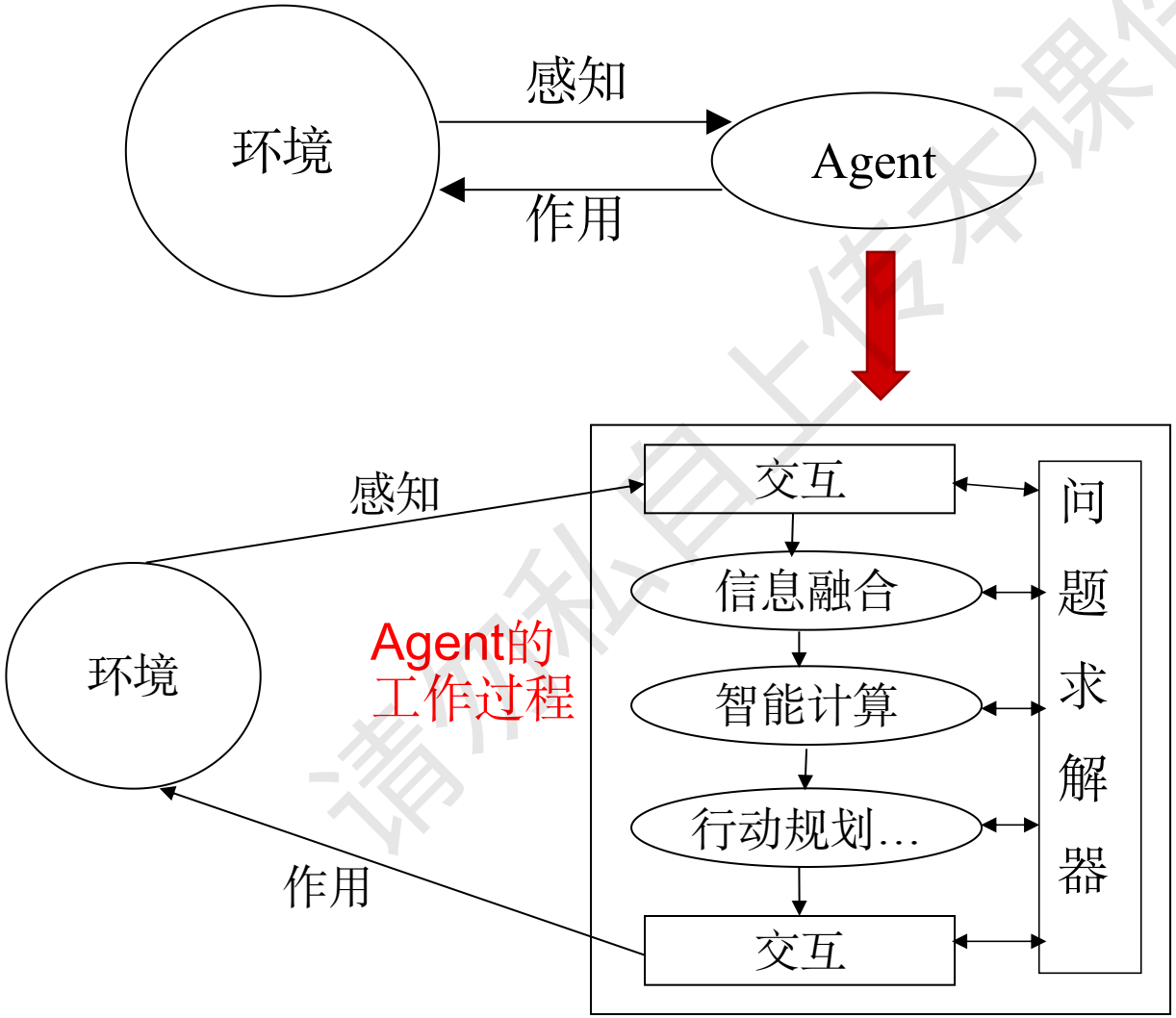
BDI-Interpreter
initialize-state();
do
    options := option-generator(event-queue, B, G, I);
    selected-options := deliberate(options, B, G, I);
    update-intentions(selected-options, I);
    execute(I);
    get-new-external-events();
    drop-successful-attitudes(B,G,I);
    drop-impossible-attitudes(B,G,I);
until quit

BDI解释器(Interpreter)

请勿私自上传本课件

1.2 Agent结构设计

- Agent体系结构



问题求解器——Agent 的‘核心引擎’

- 不直接处理原始数据，而是协调各个模块；
- 在每个阶段都参与决策支持；
- 可以是一个统一的 AI 模块（如大模型），也可以是多个子系统协作。

请勿私自上传本课件

1.2 Agent结构设计

Agent体系结构

根据人类思维的不同层次，可把Agent架构分为下列几类：

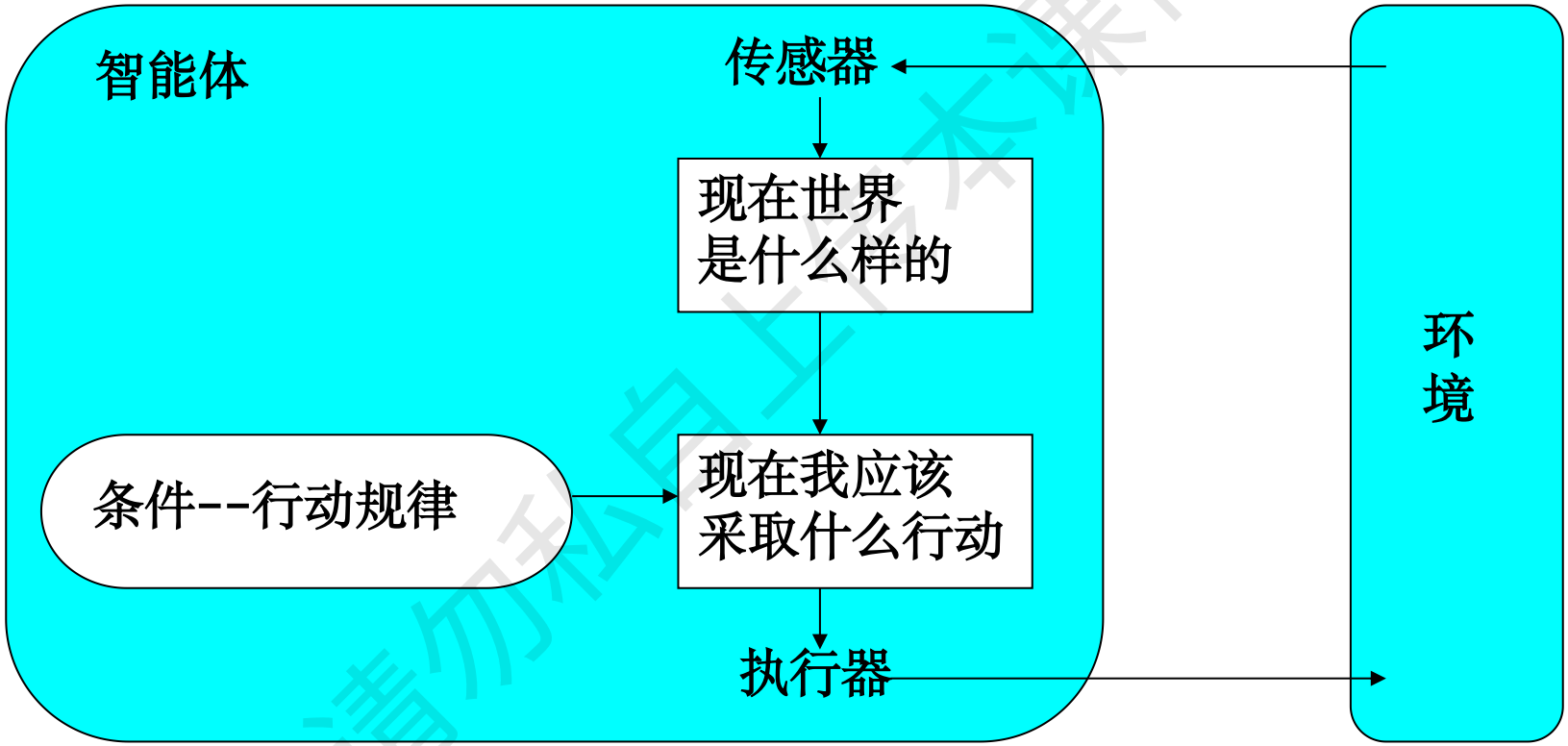
- ✓ 简单反射型智能体
- ✓ 基于模型的反射型智能体
- ✓ 基于目标的智能体
- ✓ 基于效用的智能体
- ✓ 复合式智能体

请勿私自上传本课件

1.2 Agent结构设计

- **简单反射型智能体（最基础）**

基于当前感知直接映射到动作（if-then 规则），无内部状态



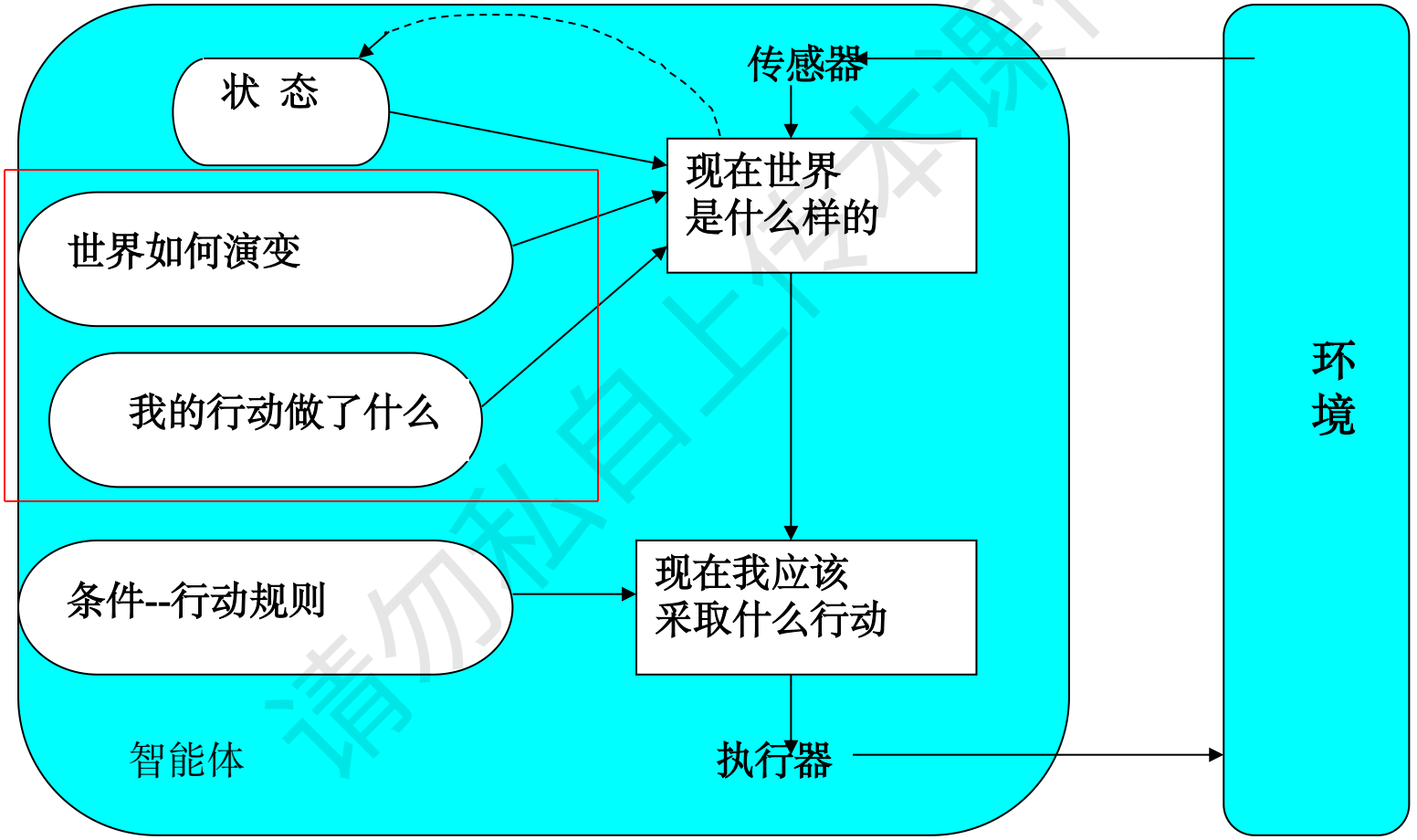
$f: P \rightarrow A$ (IF-THEN)

请勿私自上传本课件

1.2 Agent结构设计

- **基于模型的反射型智能体（升级版）**

维护内部状态，能处理部分可观测环境



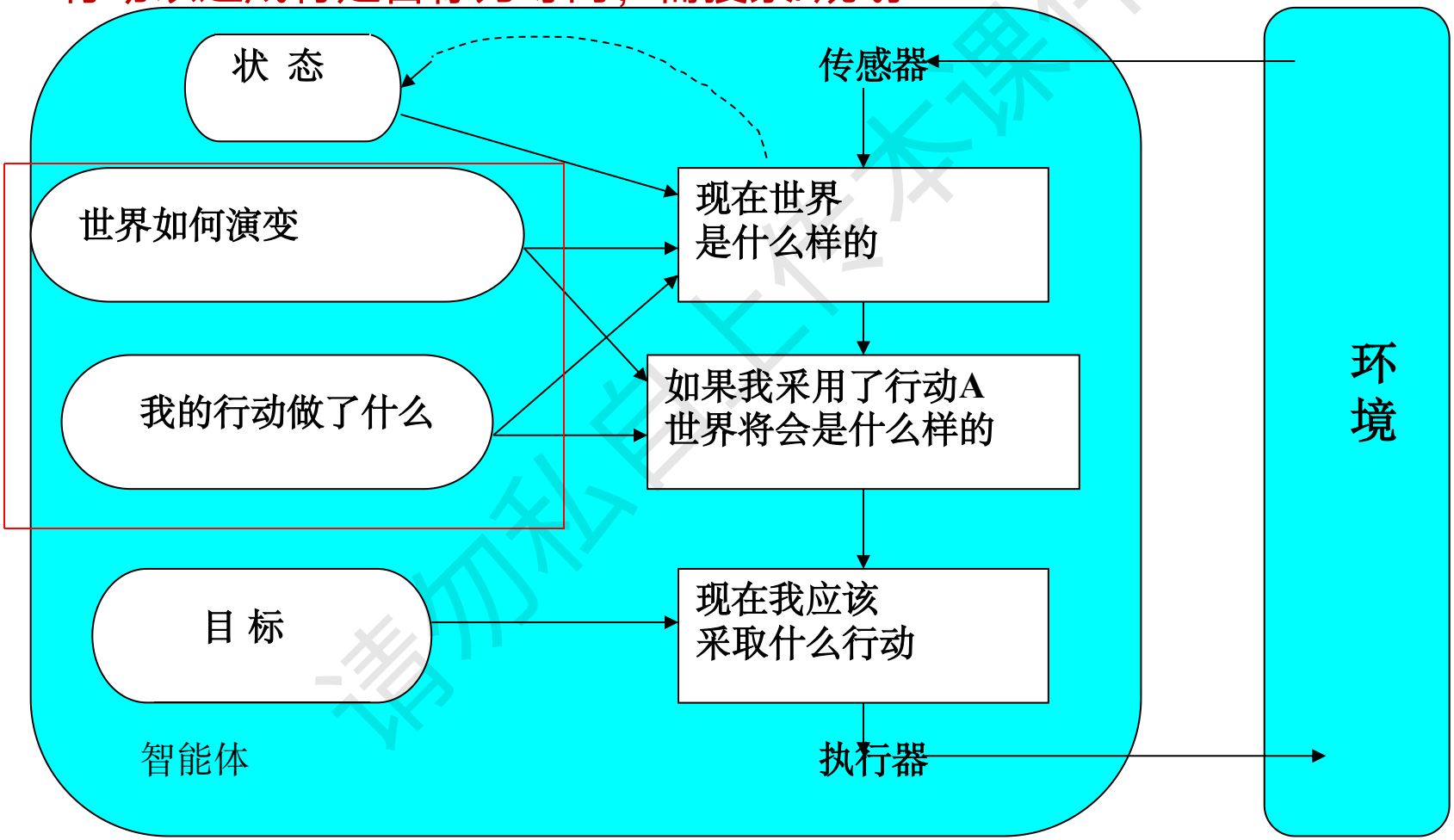
$f: P+M \rightarrow A$ (IF$^+$-THEN)

请勿私自上传本课件

1.2 Agent结构设计

- **基于目标的智能体（再升级）**

行动以达成特定目标为导向，需搜索/规划

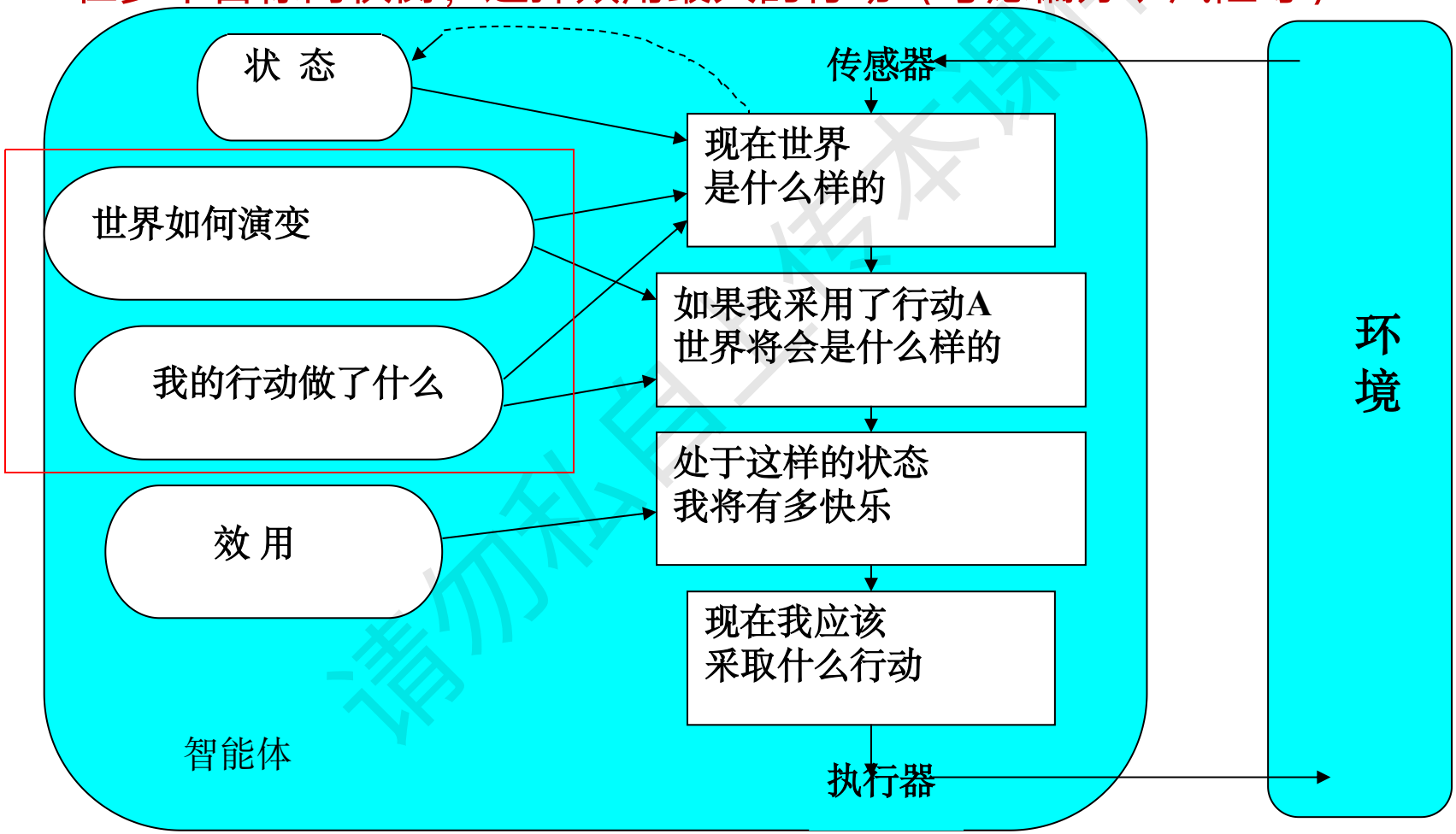


$f: P+M+Try \rightarrow A$ (Target-Try)

1.2 Agent结构设计

- **基于效用的智能体（终极进化）**

在多个目标间权衡，选择效用最大的行动（考虑偏好、风险等）

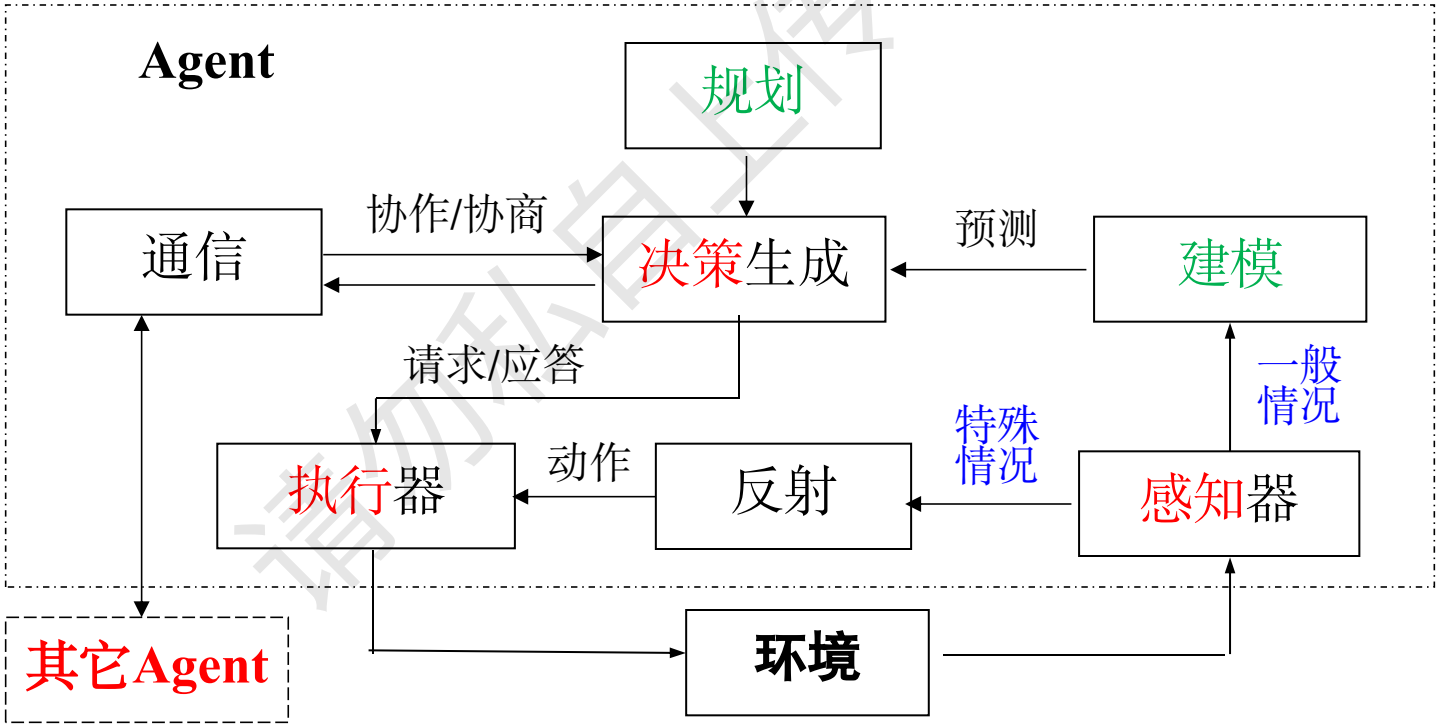


$$f: P+M+Try+Utility \rightarrow A$$

1.2 Agent结构设计

- **复合式智能体（集大成者，目前AI系统的真实架构）**

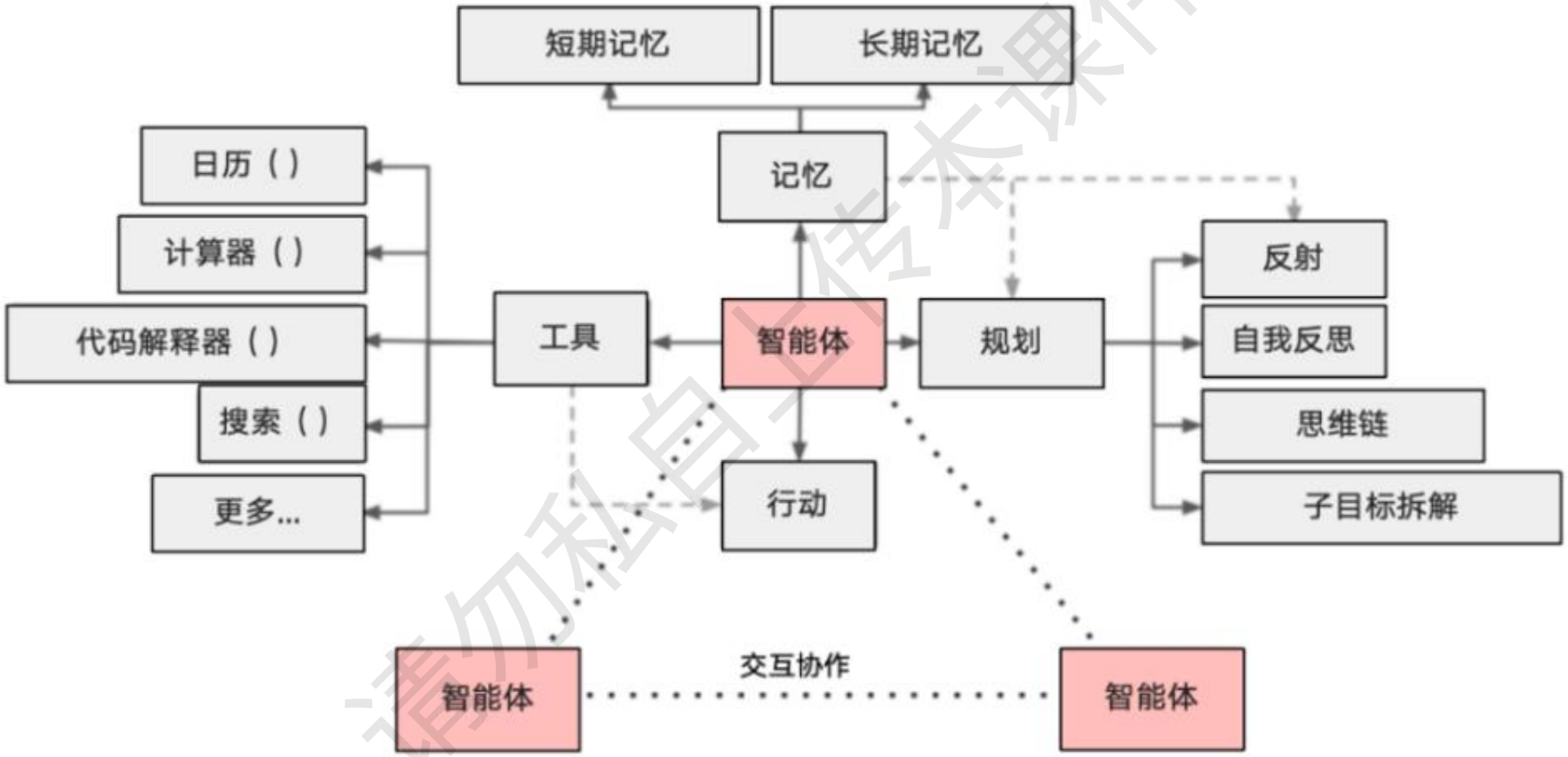
复合式Agent即在一个Agent内组合多种相对独立和并行执行的智能形态，其结构包括感知、动作、反应、建模、规划、通信和决策等模块。



请勿私自上传本课件

1.2 Agent结构设计

- 面向大模型的智能体

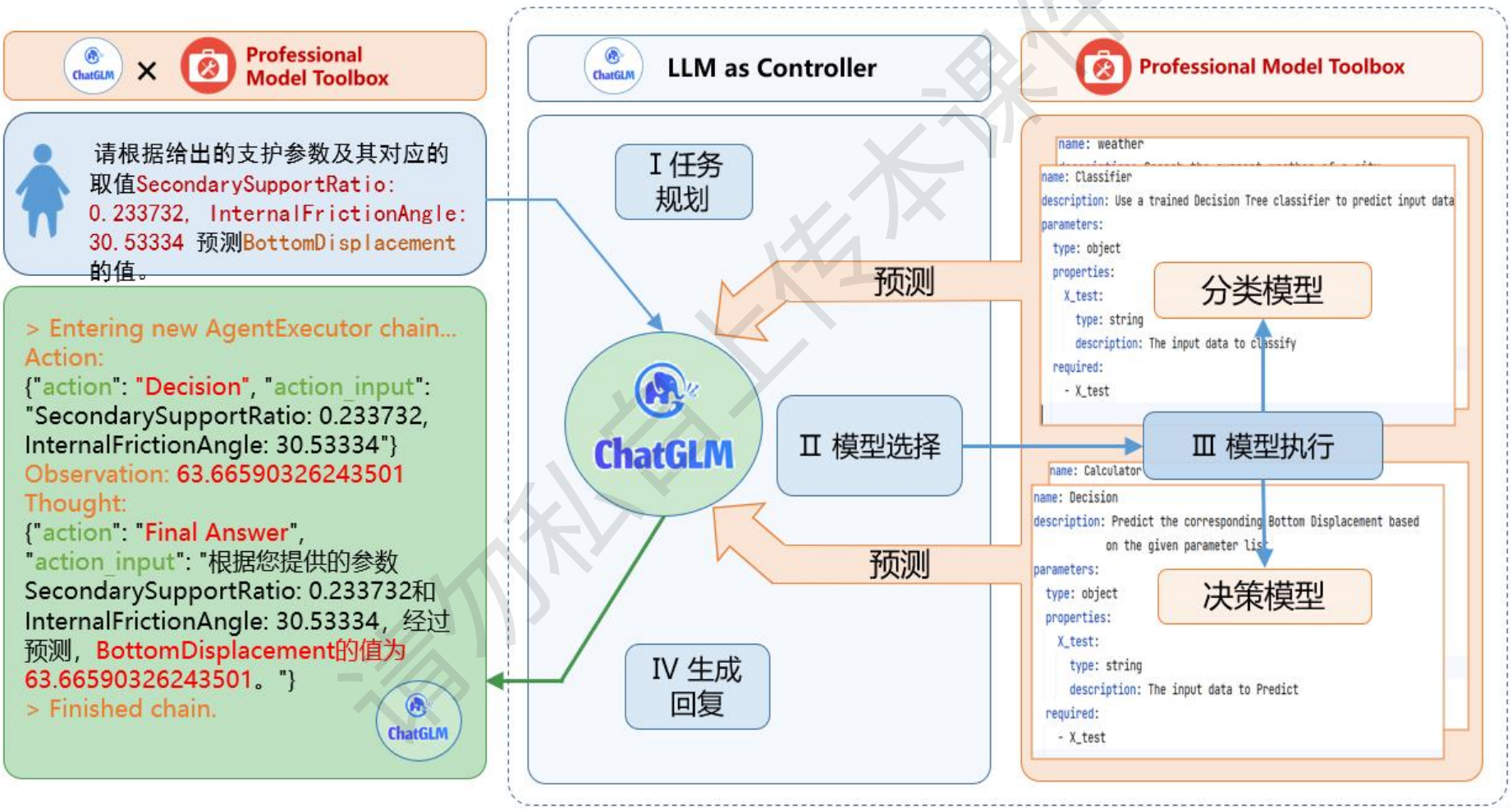


以大模型为核心驱动，具有自主理解感知、规划、记忆和使用工具的能力，能自动化执行完成复杂任务的系统（OpenAI）

请勿私自上传本课件

1.2 Agent结构设计

面向大模型的智能体



面向智能建造大模型的智能体设计

Agent与深度强化学习

1.1 什么是Agent

1.2 Agent结构设计

1.3 强化学习概述

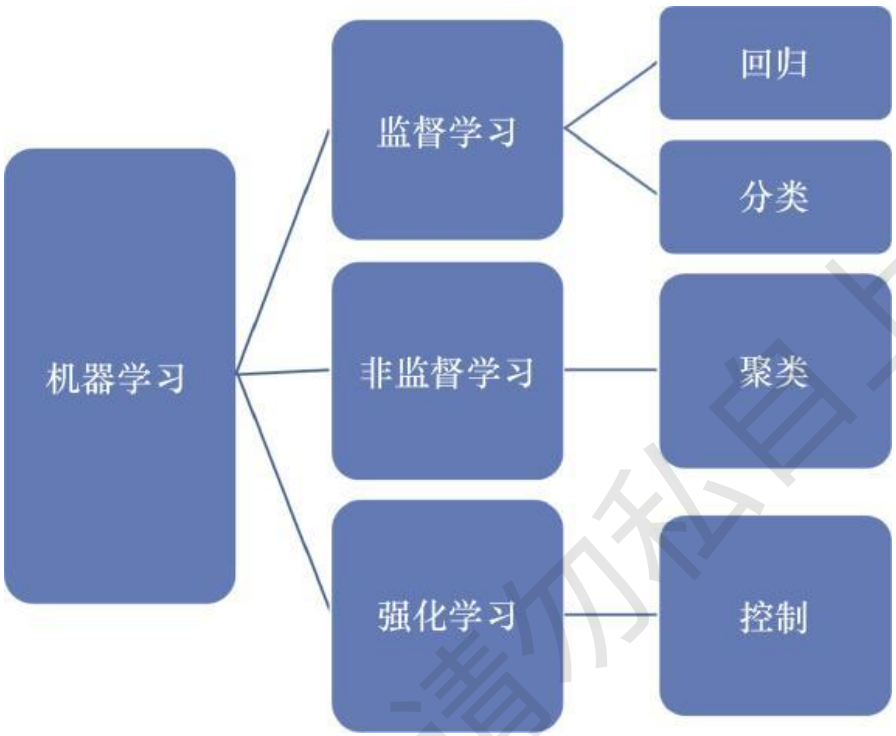
1.4 深度强化学习

1.5 应用案例分析

请勿私自上传本课件

1.3 强化学习概述

机器学习回顾　　机器学习分类

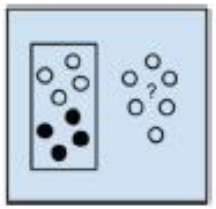


- 监督学习：通过学习从输入到输出的映射关系来进行模型训练，训练数据样本含有标签，即每个输入样本都有与之对应的输出相关联。
- 无监督学习：从未标记的数据中学习数据的结构、模式或分布，训练数据不包含目标变量。
- 强化学习：通过观察环境、执行动作并从奖励中学习，旨在使智能体在环境中学会采取一系列动作，以最大化累积奖励。

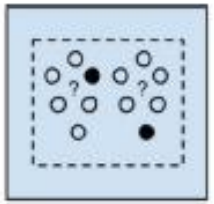
请勿私自上传课件

1.3 强化学习概述

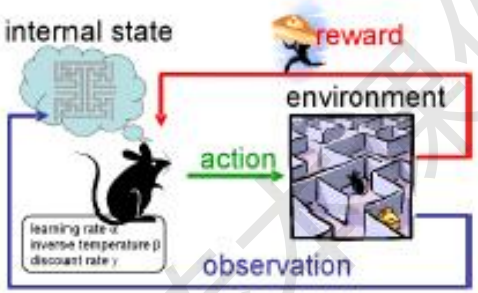
强化学习vs.机器学习



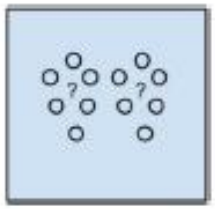
Supervised Learning



Semi-Supervised Learning



Reinforcement Learning



Unsupervised Learning



强化学习与监督学习的区别?

- ✓ 没有监督者，只有奖励/惩罚信号
- ✓ 反馈是延迟的，不是顺时的
- ✓ 时序性强，不适用于独立分布数据
- ✓ Agent行为会影响后续信息的接收

1.3 强化学习概述

强化学习的提出

- 1953到1957年，Bellman提出了求解最优控制问题的一个有效方法动态规划（dynamic programming）。
- 1957年，Bellman还提出了最优控制问题的随机离散版本，就是著名的马尔可夫决策过程（Markov decision processe, MDP）
- 1960年，Howard提出马尔可夫决策过程的策略迭代方法，这些都成为现代强化学习的理论基础。
- 1989年，Watkins提出了Q-Learning，也把强化学习的三条主线扭在了一起。
- 2016年，Deep Mind用深度强化学习设计实现AlphaGo，人机对战打败李世石。

请勿私自上传本课件

1.3 强化学习概述

■ 强化学习背景

强化思想最先来源于心理学的研究。1911年Thorndike提出了效果律（Law of Effect）：**一定情景下让动物感到舒服的行为，就会与此情景增强联系（强化），当此情景再现时，动物的这种行为也更易再现**；相反，让动物感觉不舒服的行为，会减弱与情景的联系，此情景再现时，此行为将很难再现。换个说法，哪种行为会“记住”，会与刺激建立联系，取决于行为产生的效果。



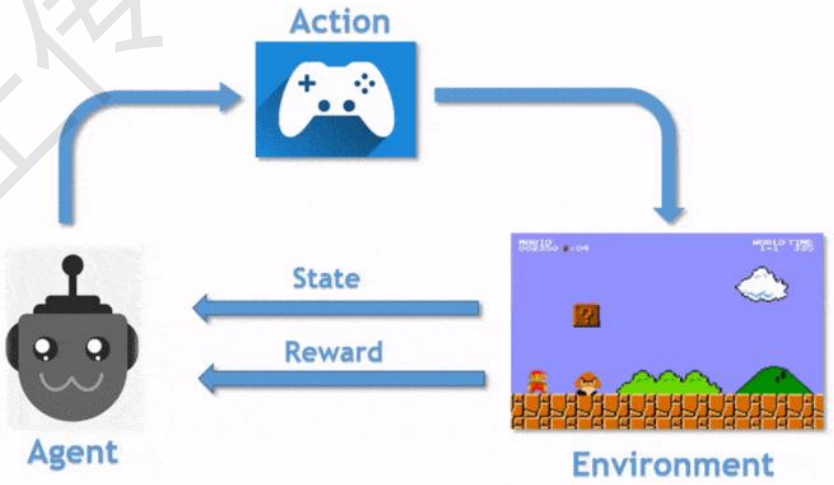
请勿私自上传本课件

1.3 强化学习概述

强化学习思想

人类通常从与外界环境的交互中学习。所谓强化（reinforcement）学习是指从环境状态到智能体行为映射的学习，以使智能体行为从环境中获得的累积奖励值最大。

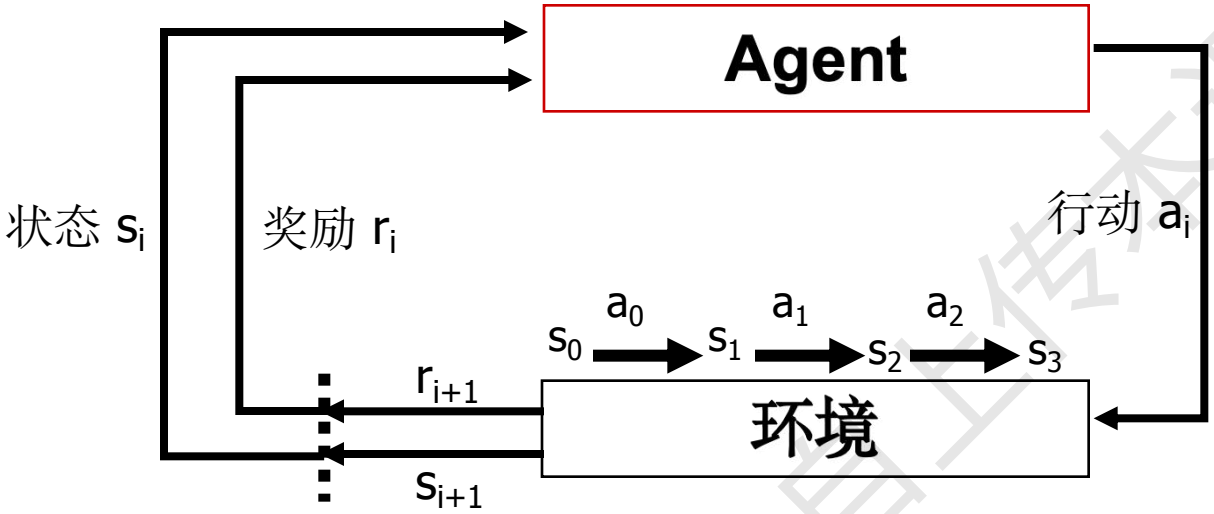
- ✓ 智能体（Agent）：感知外界环境的状态（State）和奖励反馈（Reward），并进行学习和决策。智能体的决策功能是指根据外界环境的状态来做出不同的动作（Action），而学习功能是指根据外界环境的奖励来调整策略。
- ✓ 环境（Environment）：智能体外部的所有事物，并受智能体动作的影响而改变其状态，并反馈给智能体相应的奖励。

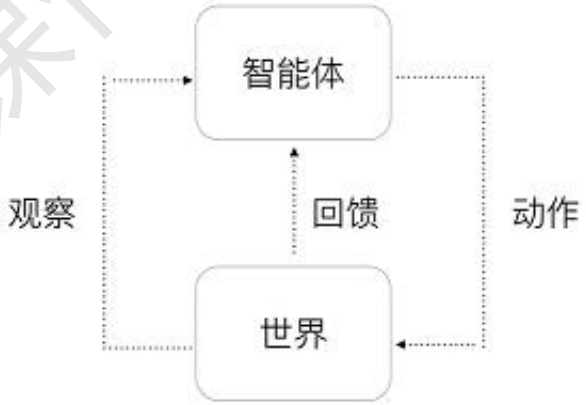


厦门理工学院 上传本课件

1.3 强化学习概述

强化学习模型





关键术语

a: action

r: reward

s: state

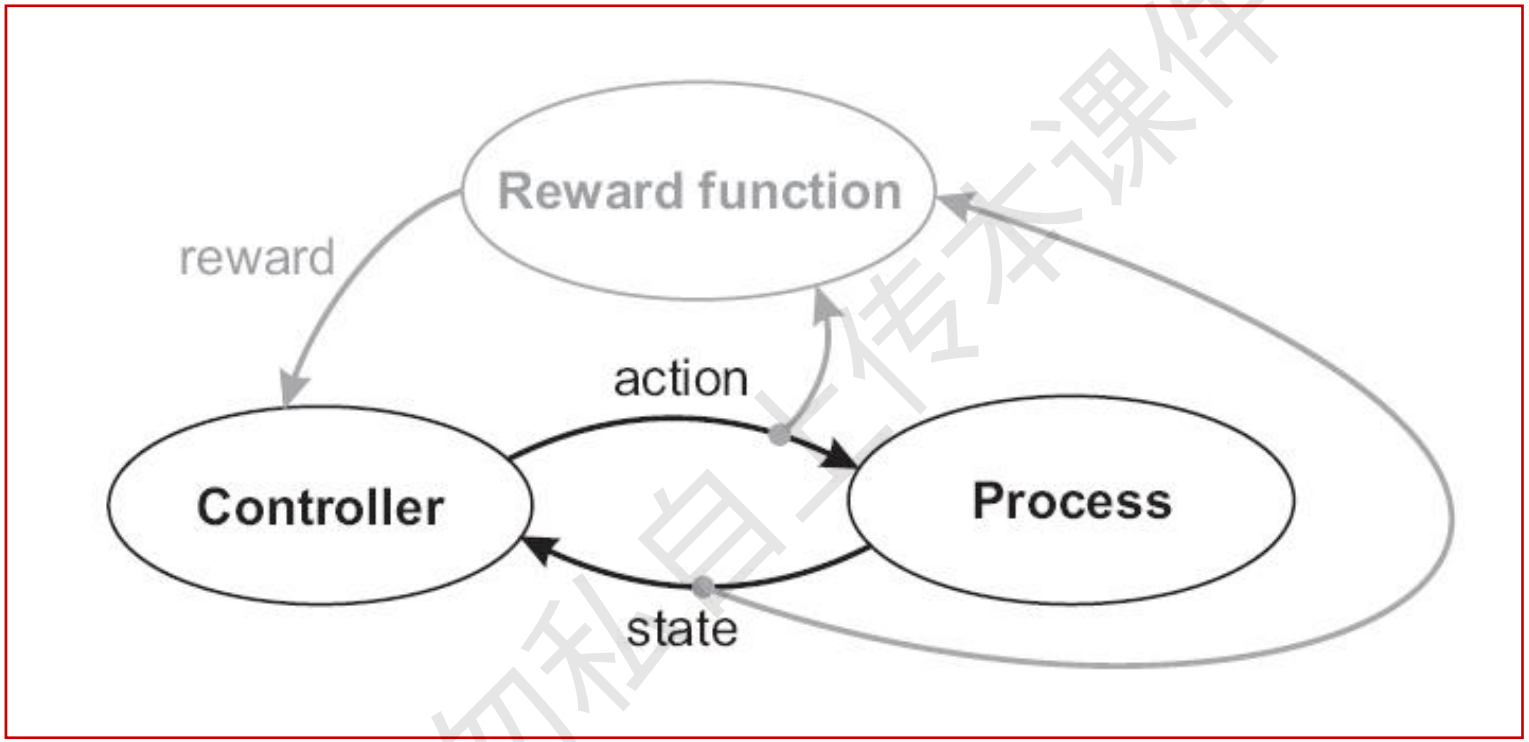
Agent与环境的交互接口包括行动（action）、奖励（reward）和状态（state）。

每一步，Agent根据策略选择一个行动执行，然后感知下一步的状态和即时奖励，通过经验再修改自己的策略。Agent的目标就是最大化长期奖励。

请勿私自上传本课件

1.3 强化学习概述

■ 强化学习模块抽象

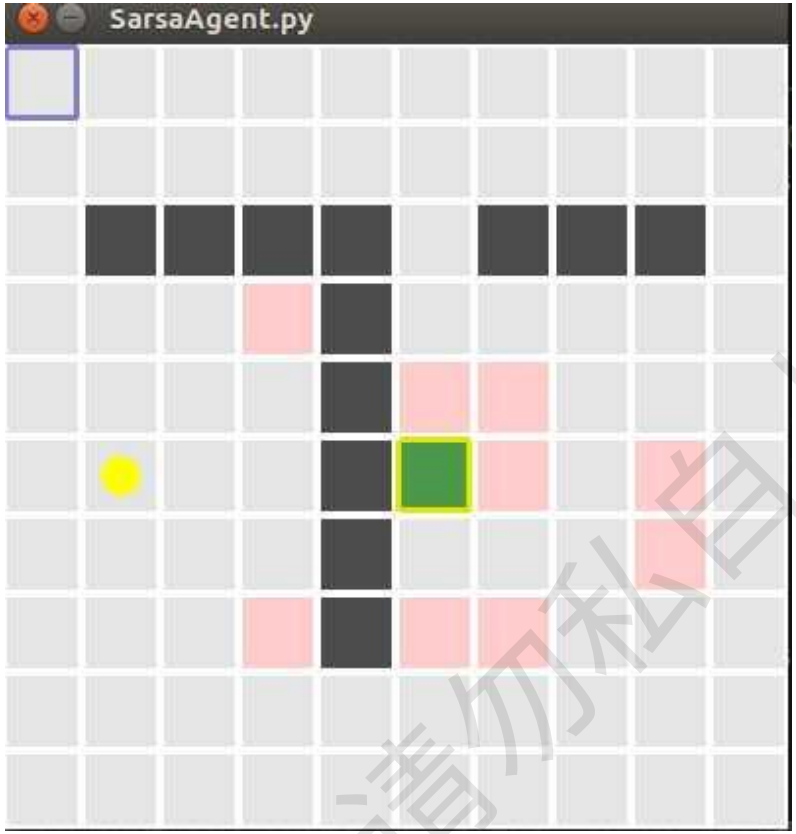


请勿私自传本课件

- ✓ 过程（Process）为控制器提供目前所处的状态。
- ✓ 控制器（Controller）根据目前的状态，为过程提供应采取的动作。
- ✓ 过程给出下一状态，并根据奖赏函数，给出其获得的奖赏（Reward）。

1.3 强化学习概述

强化学习示例

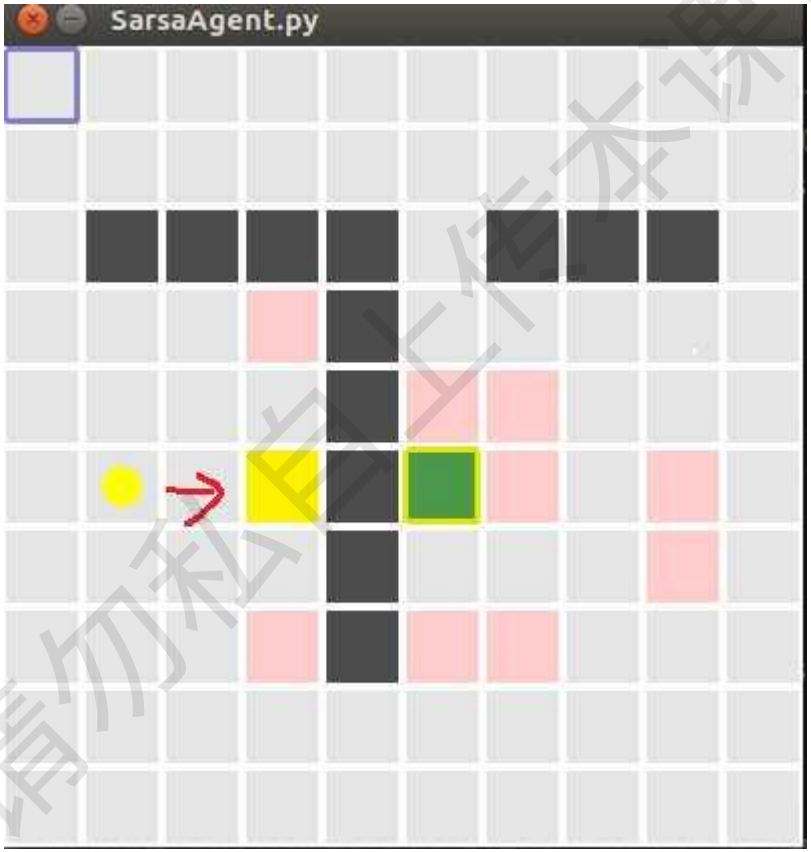


- 黄点是机器人，目的是走到绿色的方块reward+1000
- 黑色方块是墙壁，撞到reward-10
- 红色方块是陷阱，撞到reward-1000
- 其他reward+0

请勿私自上传本课件

1.3 强化学习概述

■ 强化学习示例

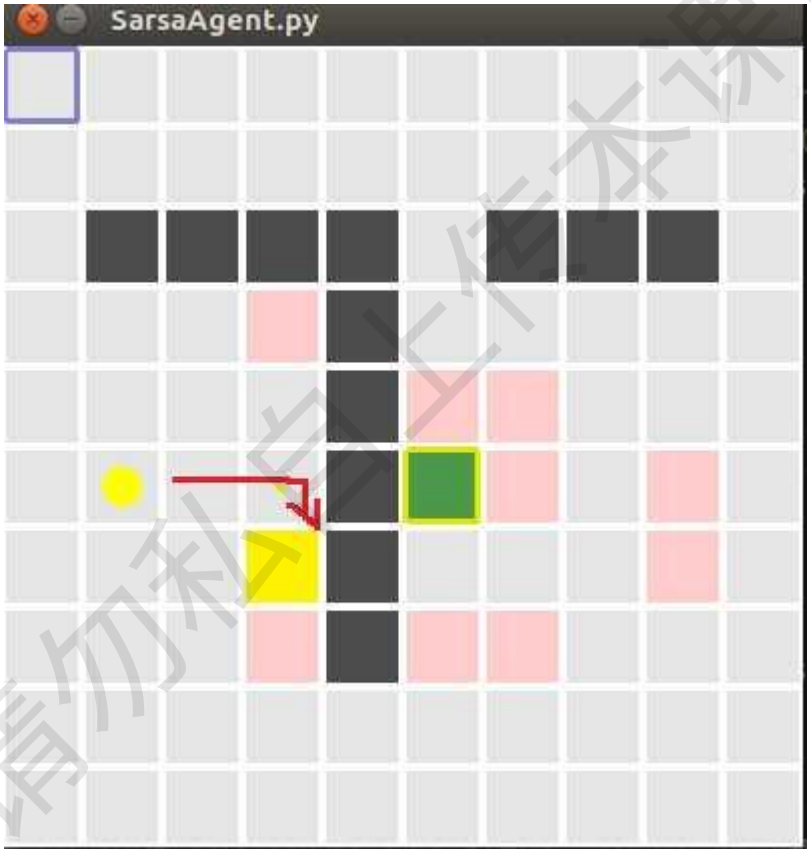


- 走到绿色终点 reward+1000
- 撞到黑色墙壁 reward-10
- 撞到红色陷阱 reward-1000
- 其他reward+0

请勿私自上传本课件

1.3 强化学习概述

强化学习示例

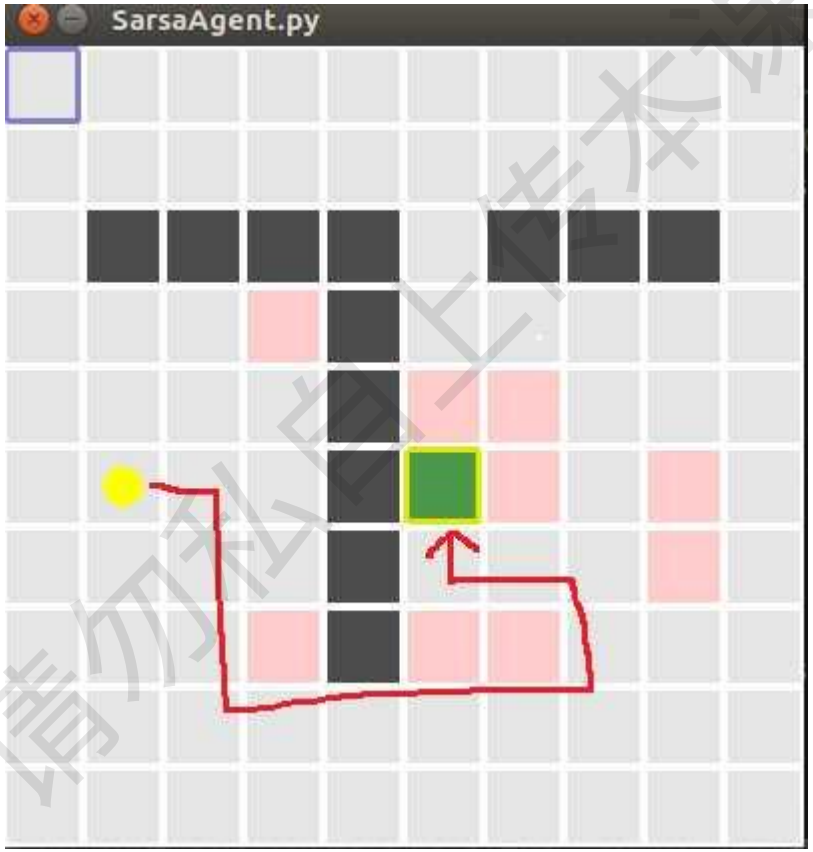


- 走到绿色终点 reward+1000
- 撞到黑色墙壁 reward-10
- 撞到红色陷阱 reward-1000
- 其他reward+0

请勿私自上传本课件

1.3 强化学习概述

强化学习示例



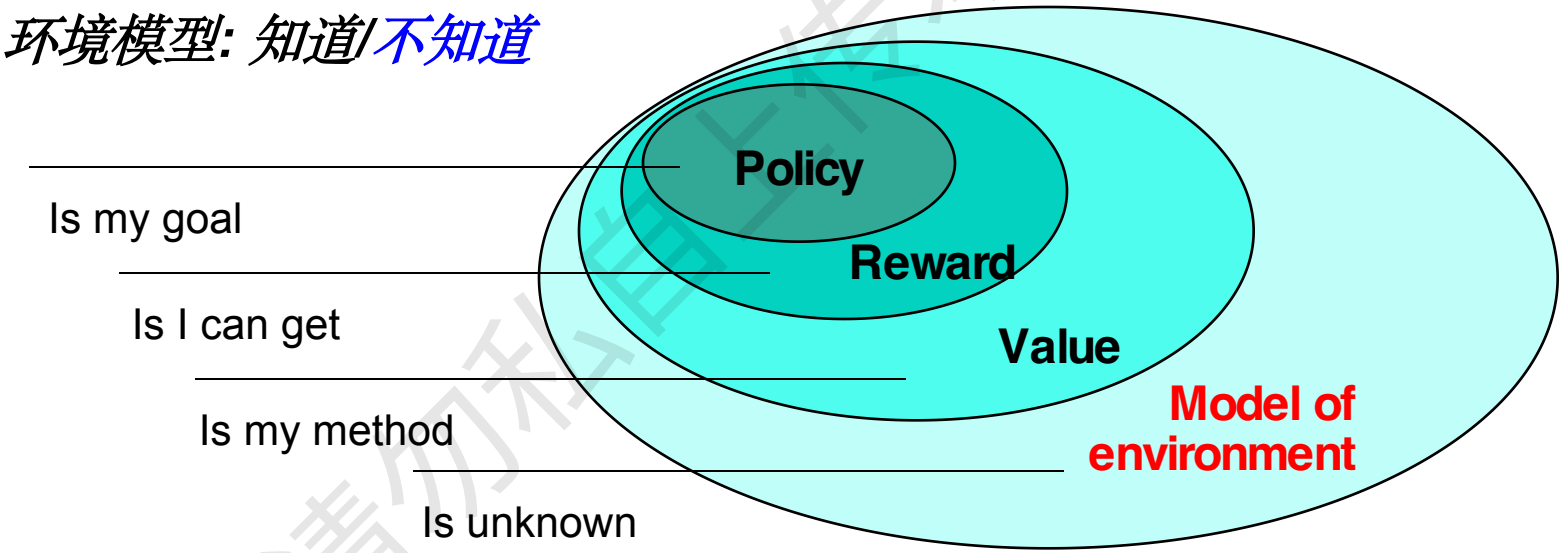
- 走到绿色终点 reward+1000
- 撞到黑色墙壁 reward-10
- 撞到红色陷阱 reward-1000
- 其他reward+0

请勿私自上传本课件

1.3 强化学习概述

强化学习原理

- ✓ ***策略:*** *随机选择动作，使得奖励最大化*
- ✓ ***奖励:*** *动作的最大化目标*
- ✓ ***价值:*** *有奖励才有价值*
- ✓ *环境模型: 知道/不知道*



强化学习是学习一个最佳策略（Policy）：使得一个智能体（Agent）应该知道在什么状态（State）下应该采取什么行动（Action），从而获取最大的回报（Reward）

请勿私自上传本课件

1.3 强化学习概述

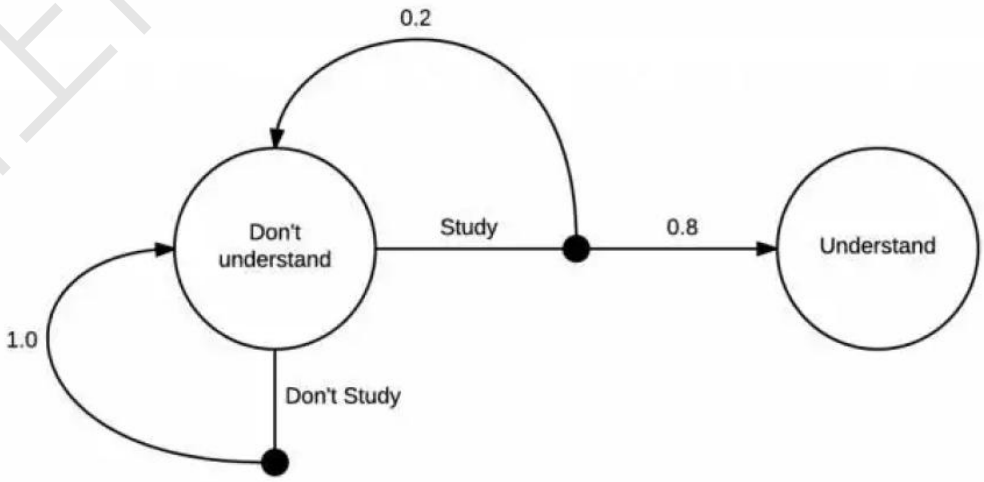
■ 强化学习原理　MDP回顾

***定义:马尔可夫决策过程(MARKOV DECISION PROCESS)**

由四元组<*S*, *A*, *R*, *P*>定义。包含：

S	A	P(s, s')	R(s, s')
状态集	动作集	转移概率	即时奖励

- ✓ **一个环境状态集*S***
- ✓ **系统行为集合*A***
- ✓ **奖励函数*R*：*S*× *A*→*R***
- ✓ **状态转移函数*P*：*S*× *A*→*PD*(*S*)**



记*R*(*s*, *a*, *s*')为系统在状态*s*采用*a*动作使环境状态转移到*s*'获得的瞬时奖励值；

记*P*(*s*, *a*, *s*')为系统在状态*s*采用*a*动作使环境状态转移到*s*'的概率。

请勿私自上传大课件

1.3 强化学习概述

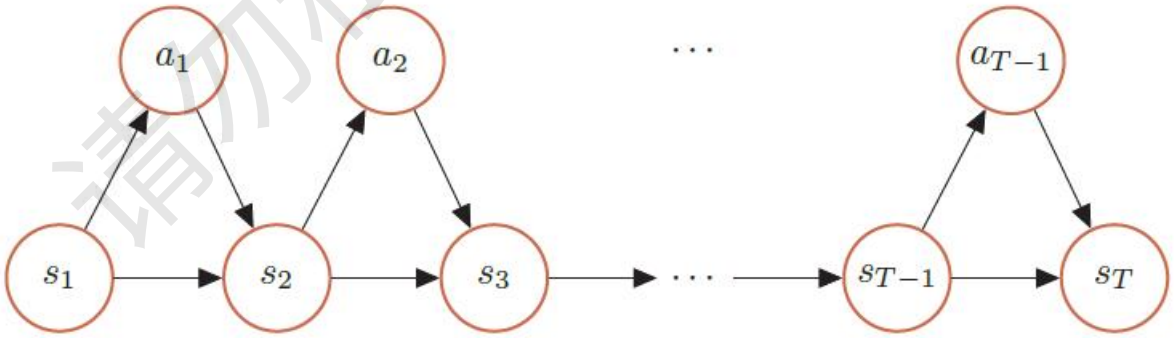
强化学习原理　MDP回顾

- ✓ 状态和动作集合: **S and A**
- ✓ 状态转移概率计算:

$$P_{ss'}^{a} = \Pr\left\{ s_{t+1} = s' \,\middle|\, s_t = s, a_t = a \right\} \quad \text{for all } s, s' \in S,\ a \in A(s).$$

- ✓ 期望奖励计算（*r*为单次瞬时奖励）：

$$R_{ss'}^{a} = E\left\{ r_{t+1} \,\middle|\, s_t = s, a_t = a, s_{t+1} = s' \right\} \quad \text{for all } s, s' \in S,\ a \in A(s).$$



请勿私自上传本课件

1.3 强化学习概述

■ 强化学习原理　核心要素

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \ldots + \gamma^{T-t-1} r_T$$

✓ 回报-Return

- □ 从时间 t 开始的未来所有奖励的加权和
- □ γ确定了延迟奖励和瞬时奖励的相对比例（折扣因子）
- □ G_t是长期性能指标

$$V^t(s) = \mathbb{E}[G_t \mid s_t = s]$$
$$= \mathbb{E}\left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots + \gamma^{T-t-1} r_T \mid s_t = s\right]$$

✓ 价值-Value

强化学习的目标是最大化期望回报 **E[G_t]**

$$V(s) = \mathbb{E}[G_t \mid s_t = s]$$
$$= \mathbb{E}\left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots \mid s_t = s\right]$$
$$= \mathbb{E}[r_{t+1} | s_t = s] + \gamma \mathbb{E}\left[r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} + \ldots \mid s_t = s\right]$$
$$= R(s) + \gamma \mathbb{E}[G_{t+1} | s_t = s]$$
$$= R(s) + \gamma \mathbb{E}[V(s_{t+1}) | s_t = s]$$
$$= R(s) + \gamma \sum_{s' \in S} p(s' \mid s) V(s')$$

贝尔曼方程：当前状态与未来状态价值的迭代关系

即时奖励+后续状态的折扣价值

请勿私自上传本课件

1.3 强化学习概述

■ 强化学习原理　目标函数

$$
\begin{aligned}
V(s) &= \mathbb{E}\left[G_t \mid s_t = s\right] \\
&= \mathbb{E}\left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots \mid s_t = s\right] \\
&= \mathbb{E}\left[r_{t+1} \mid s_t = s\right] + \gamma \mathbb{E}\left[r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} + \ldots \mid s_t = s\right] \\
&= R(s) + \gamma \mathbb{E}[G_{t+1} \mid s_t = s] \\
&= R(s) + \gamma \mathbb{E}[V(s_{t+1}) \mid s_t = s] \\
&= R(s) + \gamma \sum_{s' \in S} p(s' \mid s) V(s')
\end{aligned}
$$

强化学习的目标是学习到一个最佳策略π^*来最大化期望回报（价值函数V*），即希望智能体执行一系列的动作来获得最多的平均回报。

✓ **策略-Policy**　$\pi(a \mid s) = p(a_t = a \mid s_t = s)$

策略定义了在某状态应该采取什么动作。输入状态s，输出动作概率，对概率分布进行采样，可得到智能体将采取的动作

$$V_\pi(s) = \mathbb{E}_\pi\left[r_{t+1} + \gamma V_\pi(s_{t+1}) \mid s_t = s\right]$$

值函数：从状态s开始，执行策略π得到的期望总回报

$$V^*(s) = \max_\pi V_\pi(s)$$

V*最佳价值函数
π*最佳策略

$$\pi^*(s) = \arg\max_\pi V_\pi(s)$$

请勿私自上传本资料

1.3 强化学习概述

强化学习原理

- **动作（Action）：单次行为**

  是一个具体的、瞬时的选择

- **策略（Policy）：全局规则**

  是一个函数或映射，定义了所有状态下的动作选择方式

概念	动作（Action）	策略（Policy）
性质	具体操作	决策规则
范围	单一状态	所有状态
表示	$a \in A$	$\pi(a\|s)$ 或 $\pi(s)$
类比	一次出拳	整套拳法
强化学习中的角色	执行结果	学习目标

- **最佳策略** $\pi^*(s) = \arg\max_{\pi} V_\pi(s)$

$$V_\pi(s) = \mathbb{E}_\pi \left[r_{t+1} + \gamma V_\pi \left(s_{t+1} \right) \mid s_t = s \right]$$

$$V(s) = \mathbb{E}\left[G_t \mid s_t = s \right] = R(s) + \gamma \sum_{s' \in S} p\left(s' \mid s \right) V\left(s' \right)$$

状态转移概率

即需要获取环境模型来计算该概率

在不知道环境模型时该如何求解？

请勿私自上传本课件

1.3 强化学习概述

■ 转化：价值函数->动作价值（Q）函数

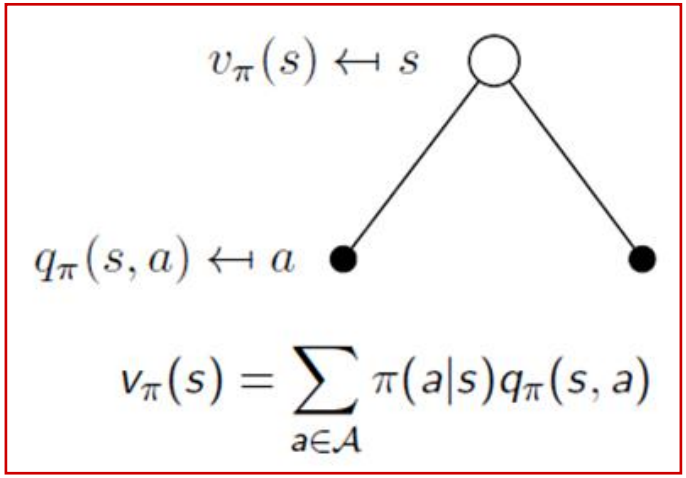
Q 函数定义：在某一个状态采取某一个动作，有可能得到的回报期望值

$$Q_\pi(s,a)=\mathbb{E}_\pi\left[G_t \mid s_t=s,a_t=a\right]$$

✓ 动作价值函数 Q-function

$$\begin{aligned}Q(s,a)&=\mathbb{E}\left[G_t \mid s_t=s,a_t=a\right]\\&=\mathbb{E}\left[r_{t+1}+\gamma r_{t+2}+\gamma^2 r_{t+3}+\ldots \mid s_t=s,a_t=a\right]\\&=\mathbb{E}\left[r_{t+1}|s_t=s,a_t=a\right]+\gamma\mathbb{E}\left[r_{t+2}+\gamma r_{t+3}+\gamma^2 r_{t+4}+\ldots \mid s_t=s,a_t=a\right]\\&=R(s,a)+\gamma\mathbb{E}[G_{t+1}|s_t=s,a_t=a]\\&=R(s,a)+\gamma\mathbb{E}[V(s_{t+1})|s_t=s,a_t=a]\\&=R(s,a)+\gamma\sum_{s'\in S}p\left(s' \mid s,a\right)V\left(s'\right)\end{aligned}$$

$$Q_\pi(s,a)=R(s,a)+\gamma\sum_{s'\in S}p\left(s' \mid s,a\right)V_\pi\left(s'\right)$$



$$Q_\pi(s,a)=R(s,a)+\gamma\sum_{s'\in S}p\left(s' \mid s,a\right)\sum_{a'\in A}\pi\left(a' \mid s'\right)Q_\pi\left(s',a'\right)$$

请勿私自上传本课件

1.3 强化学习概述

■ 经典强化学习：同策略时序差分学习法（SARSA）

算法 14.3: SARSA：一种同策略的时序差分学习算法

输入: 状态空间 $\mathcal{S}$，动作空间 $\mathcal{A}$，折扣率 γ，学习率 α

随机初始化 $Q(s,a)$;

$\forall s, \forall a, \pi(a|s) = \frac{1}{|\mathcal{A}|}$;

repeat

  初始化起始状态 s;

  选择动作 $a = \pi^{\epsilon}(s)$;

  repeat

    执行动作 a，得到即时奖励 r 和新状态 s';

    在状态 s'，选择动作 $a' = \pi^{\epsilon}(s')$;

    $Q(s,a) \leftarrow Q(s,a) + \alpha\Big(r + \gamma Q(s',a') - Q(s,a)\Big)$;

    更新策略：$\pi(s) = \arg\max_{a \in |\mathcal{A}|} Q(s,a)$;

    $s \leftarrow s', a \leftarrow a'$;

  until s 为终止状态;

until $\forall s, a, Q(s,a)$ 收敛;

输出: 策略 $\pi(s)$

来源：《神经网络与深度学习》

时序差分误差

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)]$$

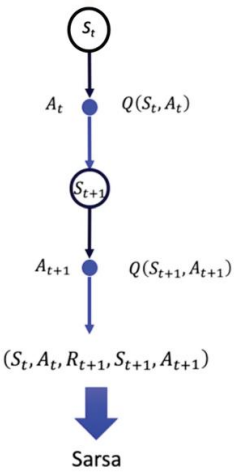
目标值　　当前值

软更新

目标值：未来收益之和

逼近

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \ldots + \gamma^{T-t-1} r_T$$



Q-Table	a_1	a_2
s_1	$Q(s_1,a_1)$	$Q(s_1,a_2)$
s_2	$Q(s_2,a_1)$	$Q(s_2,a_2)$
s_3	$Q(s_3,a_1)$	$Q(s_3,a_2)$

请勿私自上传本课件

1.3 强化学习概述

- Q-Learning（异策略时序差分方法）

算法 14.4: Q学习：一种异策略的时序差分学习算法

输入: 状态空间 $\mathcal{S}$，动作空间 $\mathcal{A}$，折扣率 γ，学习率 α

随机初始化 $Q(s,a)$;
$\forall s, \forall a, \pi(a|s) = \frac{1}{|\mathcal{A}|}$;
repeat
  初始化起始状态 s;
  repeat
    在状态 s，选择动作 $a = \pi^{\epsilon}(s)$;
    执行动作 a，得到即时奖励 r 和新状态 s';
    $Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$;
    $s \leftarrow s'$;
  until s 为终止状态;
until $\forall s, a, Q(s,a)$ 收敛;

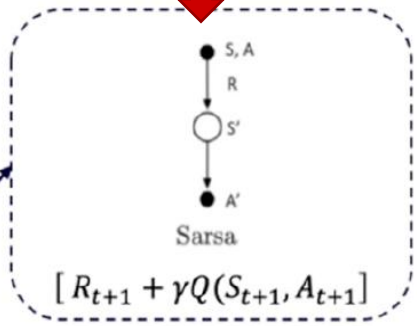
输出: 策略 $\pi(s) = \arg\max_{a \in |\mathcal{A}|} Q(s,a)$

来源：《神经网络与深度学习》

Q学习 vs. Sarsa

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[\ Q\text{目标值} - Q(S_t, A_t)]$

目标差异



Q学习算法不通过πε来选下一步的动作a′，而是直接选最优的Q函数

清华私自上传本课件

1.3 强化学习概述

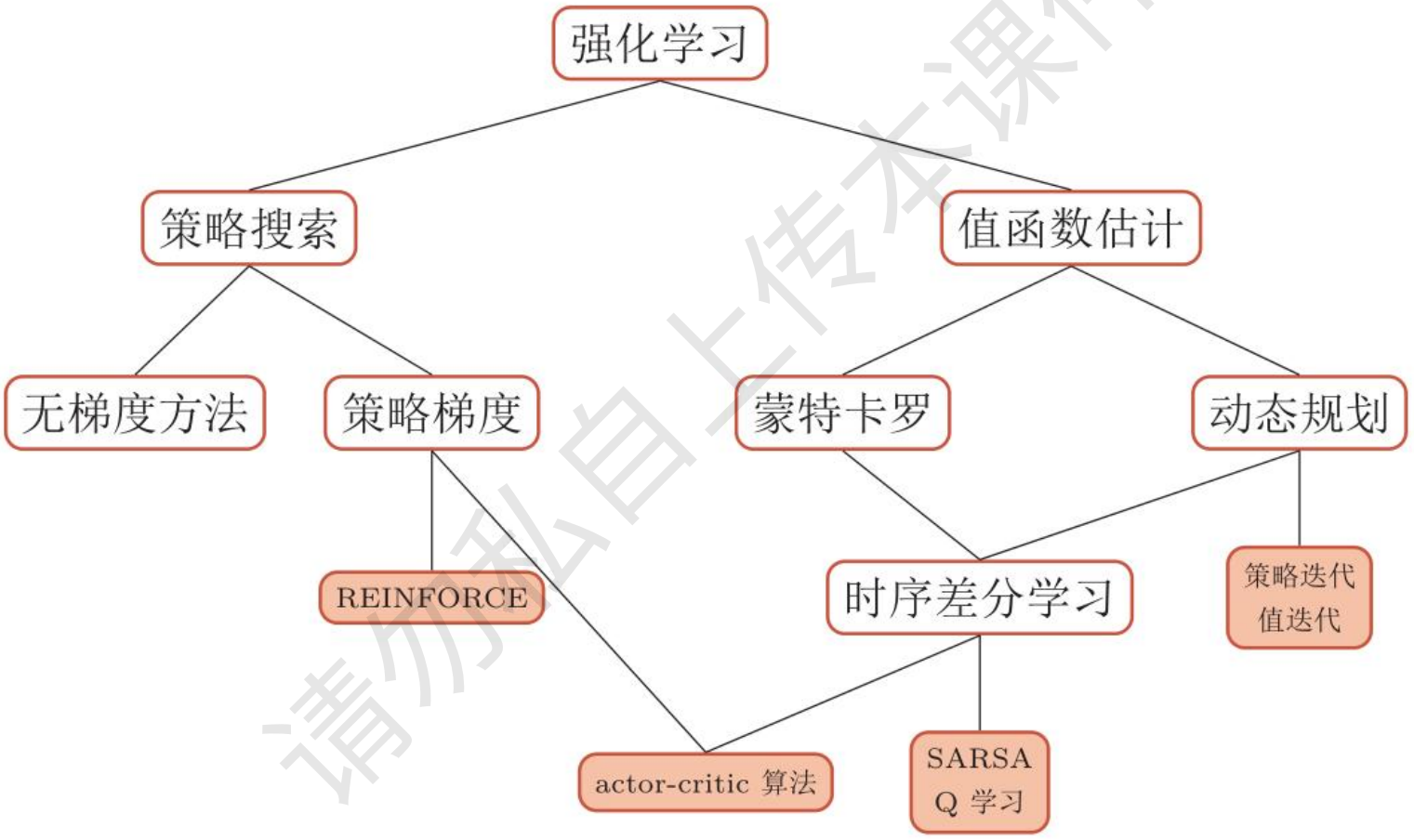
- Q-learning vs. Sarsa

维度	SARSA	Q-Learning
目标值	$r + \gamma Q(s', a')$	$r + \gamma \max_{a'} Q(s', a')$
是否使用 a'	是，使用实际选择的动作	否，使用最优动作
策略类型	On-Policy（同策略）	Off-Policy（异策略）
学习速度	较慢，保守	较快，激进
稳定性	更稳定	可能不稳定（如 DQN 中需经验回放）
是否需要知道最优动作	❌ 不需要	✅ 需要（隐式地）
是否能学习最优策略	✅ 能，但较慢	✅ 快速收敛到最优
是否支持 off-policy	❌ 不支持	✅ 支持
是否适合深度学习	✅ 可行	✅ 更常用（DQN 基础）
是否更稳健	✅ 是	⚠️ 有时过拟合

请勿私自上传本课件

1.3 强化学习概述

- **经典强化学习方法概览**



请勿私自上传本课件

1.3 强化学习概述

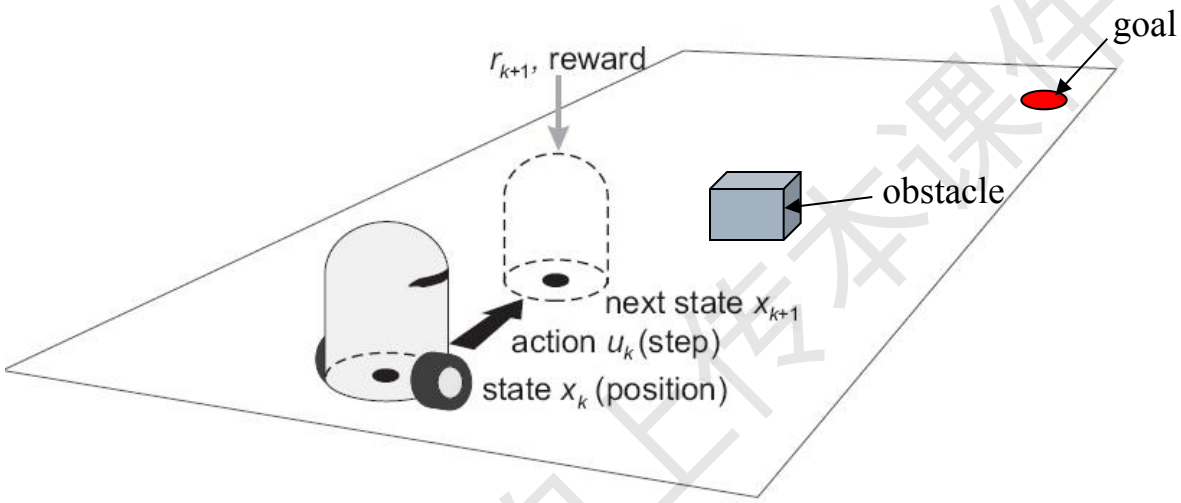
■ 经典强化学习方法概览

方法	是否需模型	是否在线学习	是否可处理连续动作	代表算法
策略搜索	❌ 不需要	✅ 是	✅ 是	REINFORCE, PPO
值函数估计	❌ 不需要	✅ 是	⚠️ 有限制	Q-learning, DQN
蒙特卡罗	❌ 不需要	❌ 否（需完整 episode）	⚠️ 有限制	Monte Carlo
动态规划	✅ 需要	❌ 否	❌ 否	Value Iteration
时序差分	❌ 不需要	✅ 是	⚠️ 有限制	SARSA, Q-learning
Actor-Critic	❌ 不需要	✅ 是	✅ 是	A2C, PPO

请勿私自上传本课件

1.3 强化学习概述

实例分析：机器人行动



目标：避开障碍物，从底端到达右上角的目标。

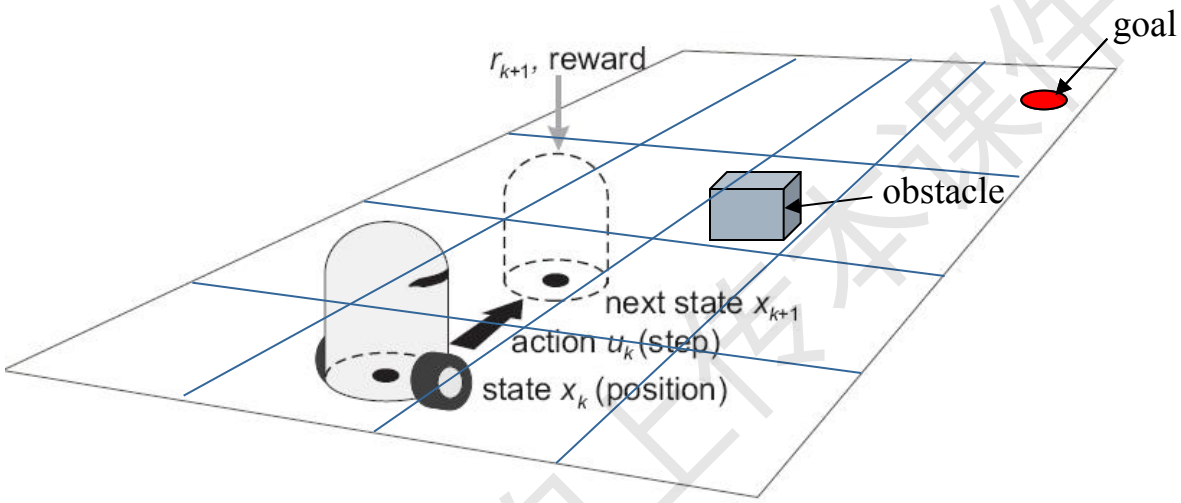
控制器（agent）:机器人的软件，决策算法。

过程（环境）：与机器人密切相关的环境（地面、障碍物、目标等）。用于决策的物理实体、传感器和执行器。

请勿私自上传本课件

1.3 强化学习概述

■ 实例分析：机器人行动



状态（x）：机器人的位置（直角坐标）。

动作（u）：机器人走一步（直角坐标）。

迁移函数（f）：从目前的位置走一步，到达下一位置，遇到障碍变复杂。

奖赏函数（ρ）：产生奖赏（r），评价迁移的质量。

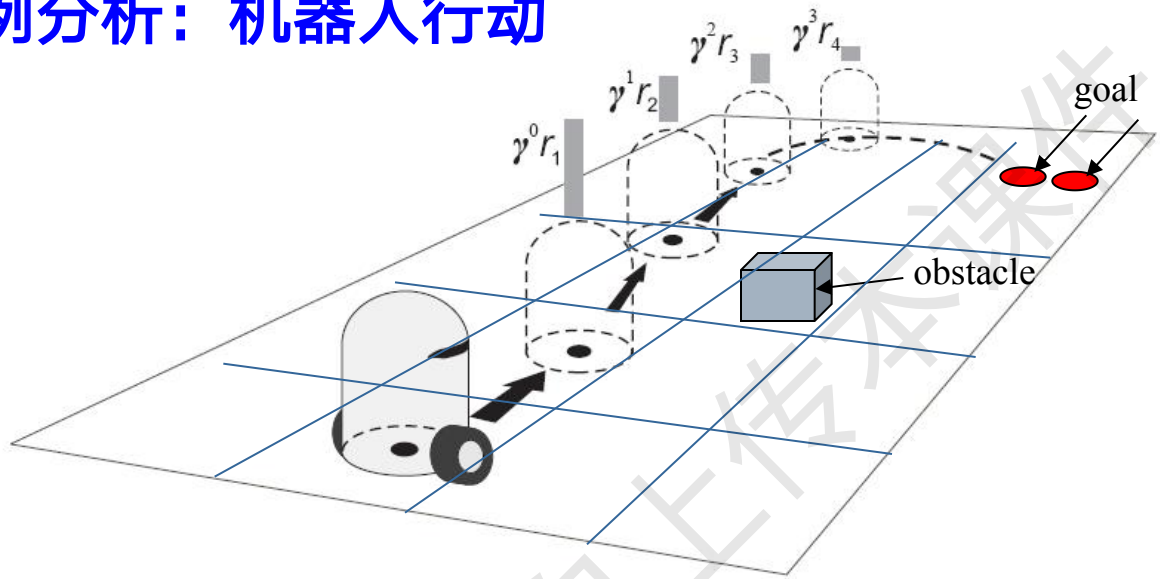
目标：+10；障碍：-1；其他：0，可构造带更多信息的奖赏。

策略（h）:从状态到动作的映射。

请勿私自上传本课件

1.3 强化学习概述

- **实例分析：机器人行动**



目标是使回报最大化，其中回报是由交互过程中的累积奖赏构成。

主要考虑折扣无限水平回报，即累积回报开始于初始时间步k=0，沿（可能）无限长的轨迹，对得到的奖赏值进行累积，通过一个因子$\gamma \in [0,1]$对奖赏加权，这个因子随着时间步的增加呈指数地减少。

$$\gamma^0 r_1 + \gamma^1 r_2 + \gamma^2 r_3 + \ldots$$

1.3 强化学习概述

实例分析：机器人行动

奖赏依赖于所遵循的状态-动作轨迹，每个奖赏r_{k+1}是迁移(x_k,u_k,x_{k+1})的结果，状态-动作轨迹依赖于使用的策略：

$$x_0,\ u_0 = h(x_0),\ x_1,\ u_1 = h(x_1),\ x_2,\ u_2 = h(x_2),\ \ldots$$

强化学习的核心挑战

得到一个解，通过由立即奖赏构成的回报优化长期性能。

解RL问题转化为找最优策略h^*问题，即对每个初始状态，使其回报最大化。

得到最优策略的方法：

计算最大的回报：

$$h^*(x) \in \arg\max_u Q^*(x,u)$$

$$Q^*(x,u) = \gamma^0 r_1 + \gamma^1 r_2 + \gamma^2 r_3 + \ldots$$

when $x_0 = x, u_0 = u$, and optimal actions are taken for $x_1, x_2, \ldots$

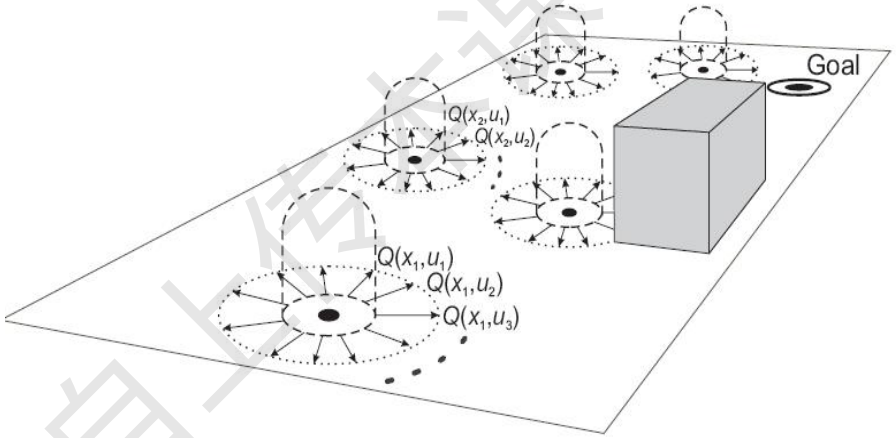
请勿私自上传本课件

1.3 强化学习概述

■ 实例分析：机器人行动

为保证表示的精确性，对每个状态-动作对，存储各自的Q函数值（Q值）。

Q-Table	a_1	a_2
s_1	$Q(s_1, a_1)$	$Q(s_1, a_2)$
s_2	$Q(s_2, a_1)$	$Q(s_2, a_2)$
s_3	$Q(s_3, a_1)$	$Q(s_3, a_2)$



思考：存在问题？

情况	是否可行	说明
离散状态空间	✅ 可行	如网格世界，可用Q-table
连续状态空间	❌ 不可行	无法穷举所有状态-动作对

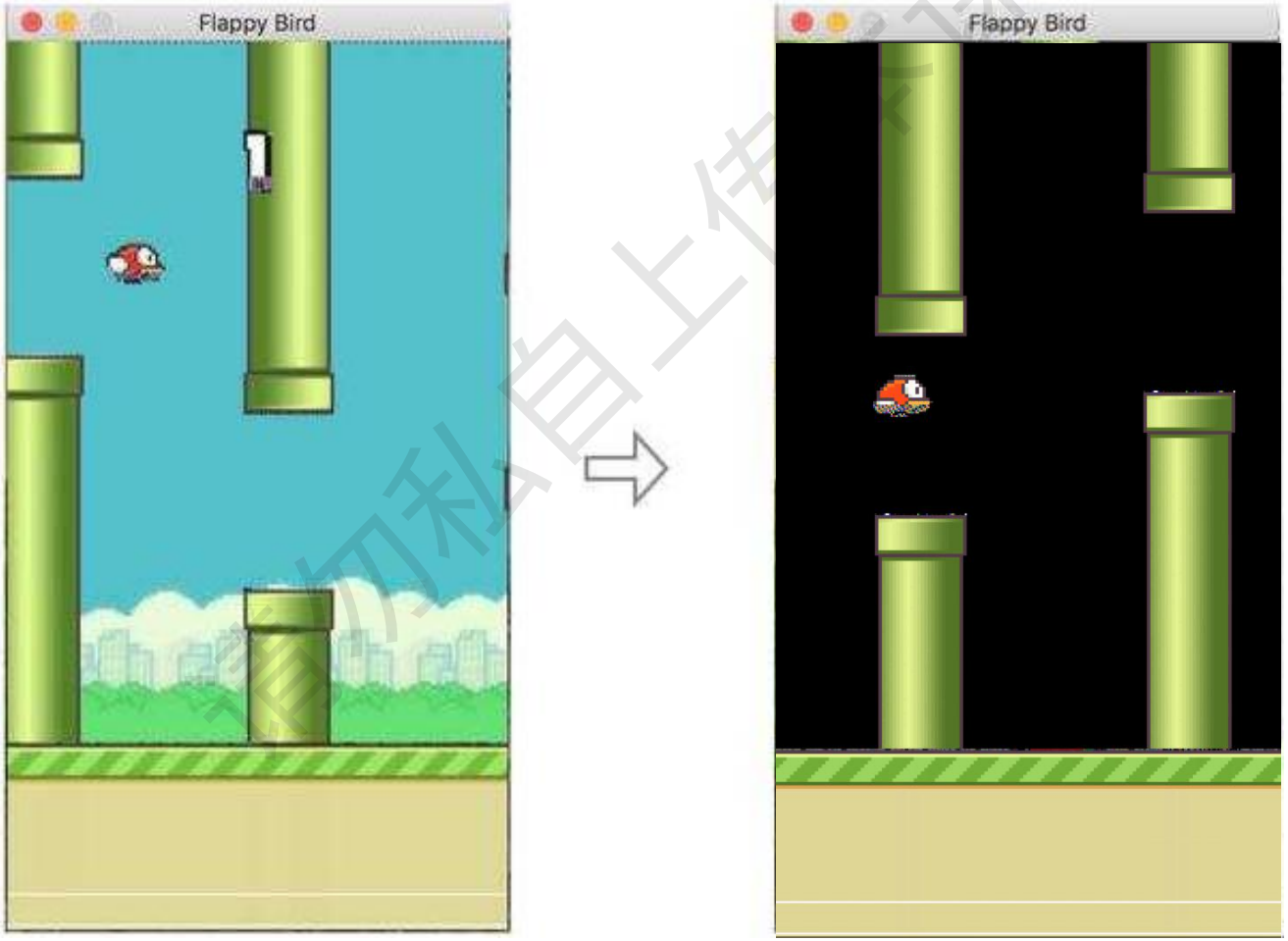
对于机器人的每个位置，以及相应位置可能采取的每个走步，都必须存储其值。但位置和走步都是连续变化的，因此可能得到无穷多个不同的值，怎么处理？

请勿私自上传课件

课堂练习

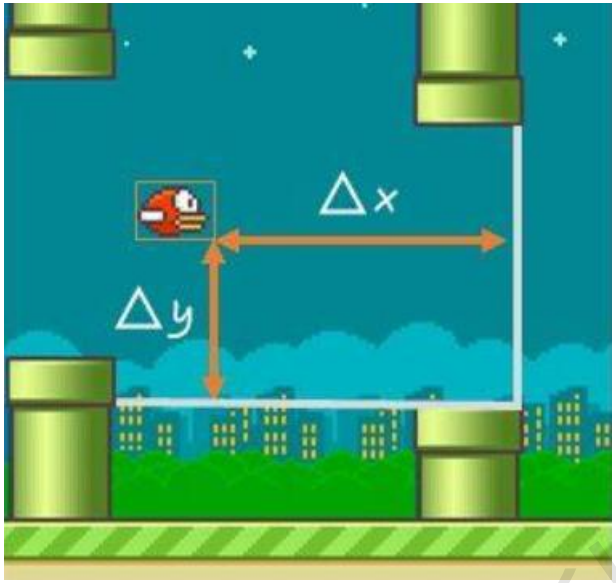
采用强化学习设计实现flappy bird游戏，

给出核心设计思路及伪代码描述



课件私自上传

课堂练习



动作
每一个状态，有两个可能的动作
•点击一下
•啥也不干
奖励
奖励的机制完全基于鸟命这个参数
•+1，如果小鸟还活着
•-1000，如果小鸟死了
•+50，成功经过一根柱子

状态	飞	不飞
$(\Delta x_1, \Delta y_1)$	1	20
$(\Delta x_1, \Delta y_2)$	20	-100
…	…	…
$(\Delta x_m, \Delta y_{n-1})$	-100	2

1. Initialize Q arbitrarily //随机初始化Q值
2. Repeat (for each episode): //每一次游戏，从小鸟出生到死亡是一个episode
3. Initialize S //小鸟刚开始飞，S为初始位置的状态
4. Repeat (for each step of episode):
5. 根据当前Q和位置S，使用ε-greedy策略，得到动作A
6. 执行动作A，小鸟到达新的位置S'，并获得奖励R
7. $Q(S,A) \leftarrow (1-\alpha)*Q(S,A) + \alpha*[R + \gamma*\max Q(S',a)]$ //在Q中更新S
8. $S \leftarrow S'$
9. until S is terminal //即到小鸟死亡为止

Agent与深度强化学习

1.1 什么是Agent

1.2 Agent结构设计

1.3 强化学习概述

1.4 深度强化学习

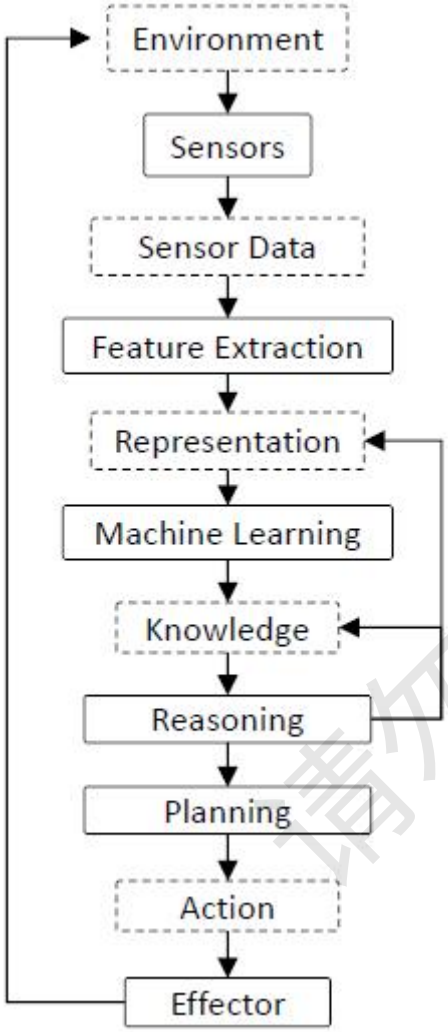
1.5 应用案例分析

请勿私自上传本课件

1.4 深度强化学习

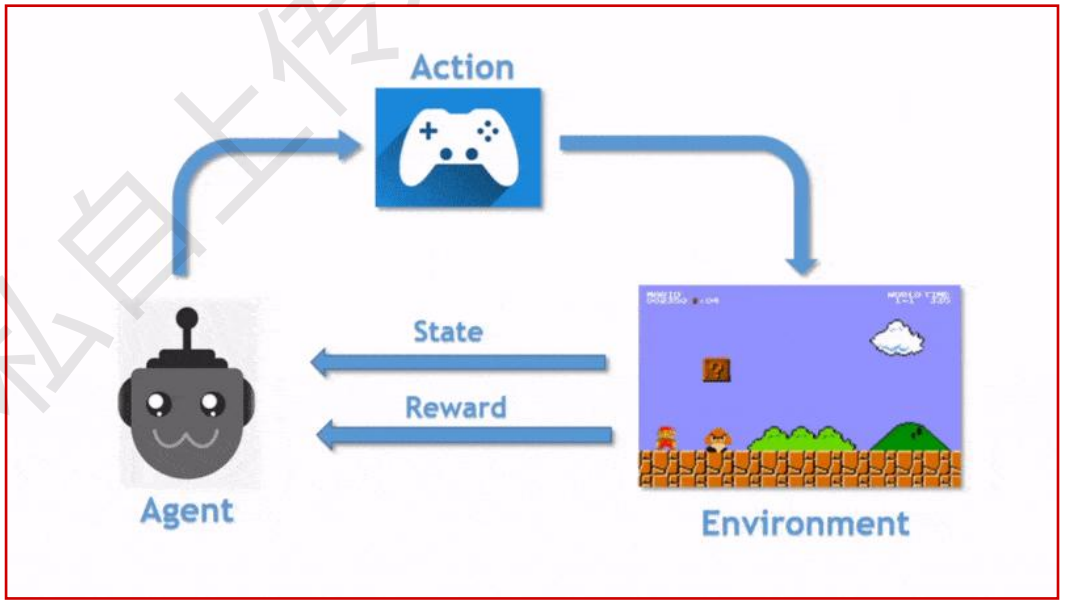
- 从深度学习到深度强化学习

智能体



为什么需要深度强化学习？

Why Deep Reinforcement Learning

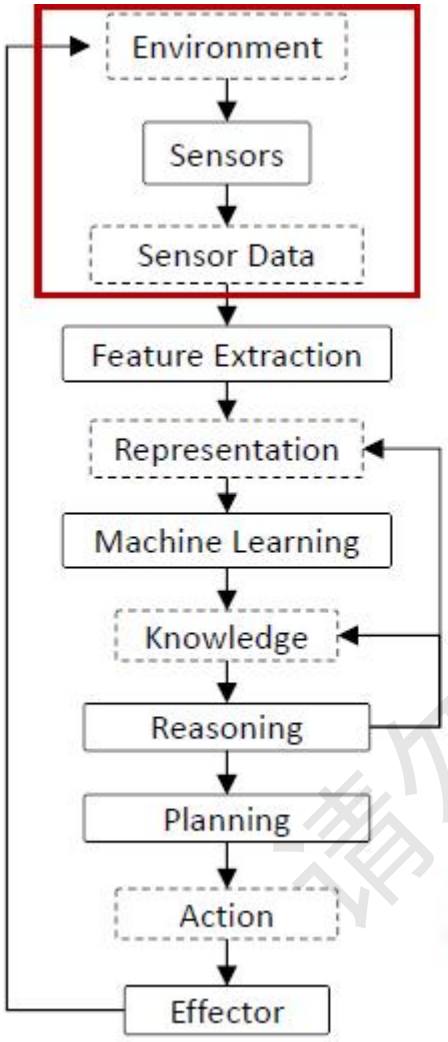


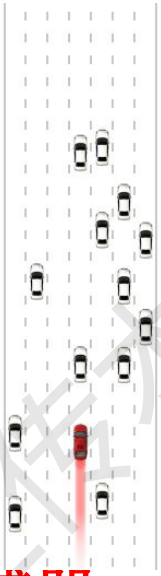
请勿私自上传本课件

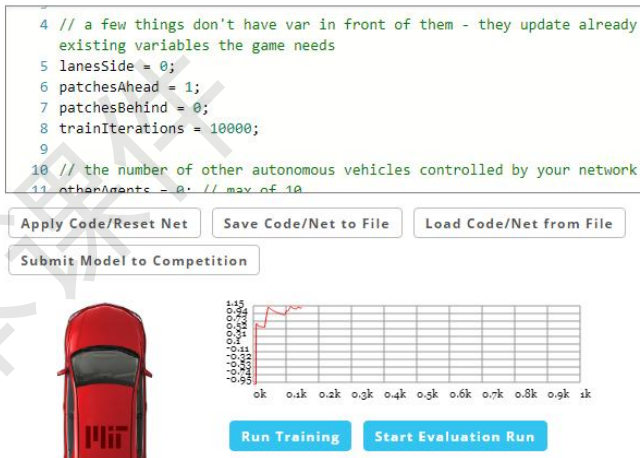
1.4 深度强化学习

- 从深度学习到深度强化学习

智能体









传感器：大数据


Lidar


Camera
(Visible, Infrared)


Radar


GPS


Stereo Camera


Microphone


Networking
(Wired, Wireless)

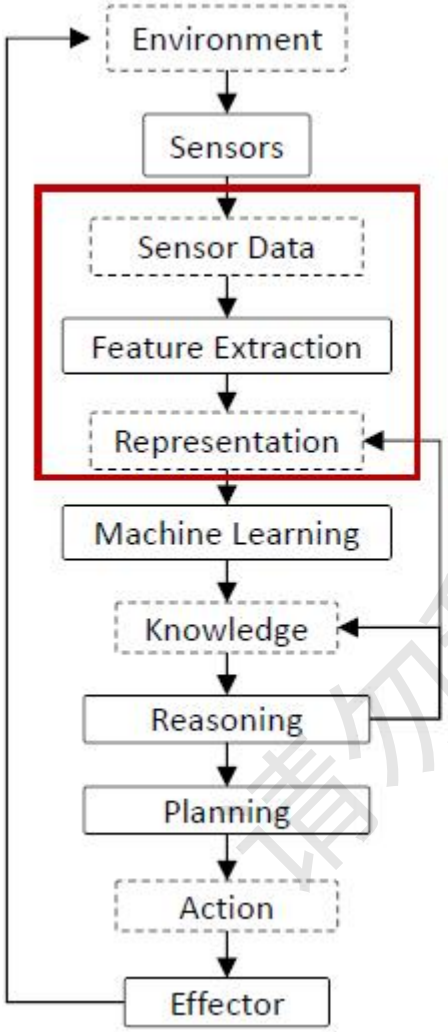

IMU

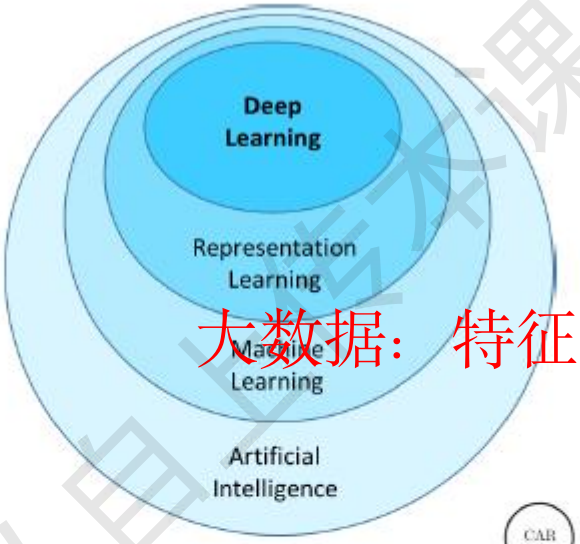
请勿私自上传课件

1.4 深度强化学习

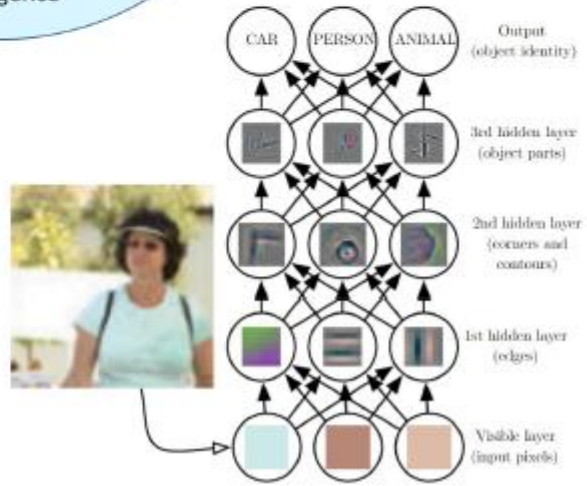
■ 从深度学习到深度强化学习

智能体





大数据：特征表示

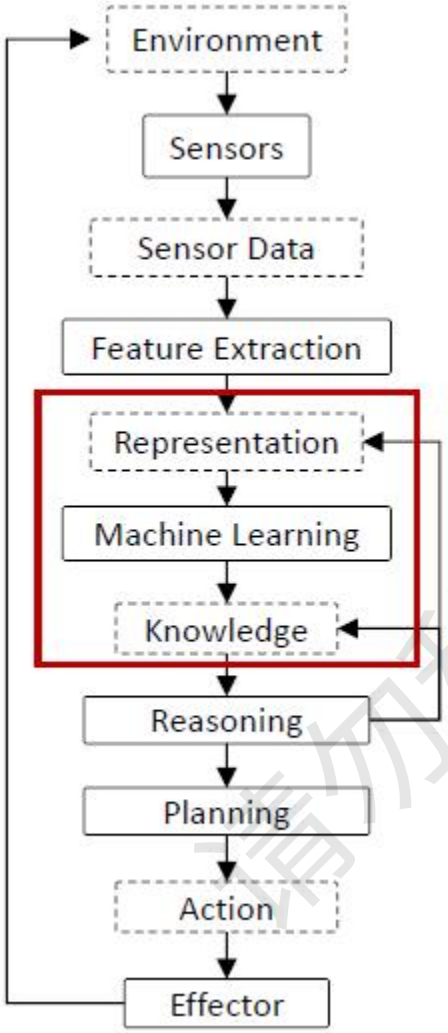


请勿私自上传课件

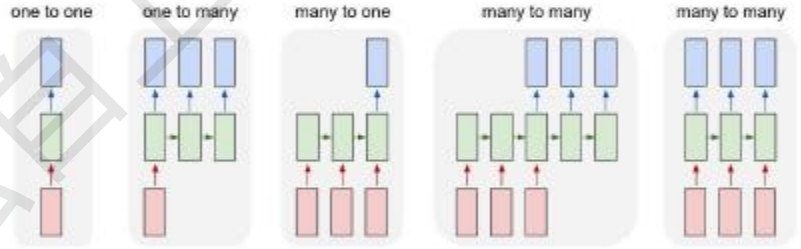
1.4 深度强化学习

■ 从深度学习到深度强化学习

智能体



深度特征：智能学习



清华私自上传本课件

1.4 深度强化学习

■ 从深度学习到深度强化学习

智能体

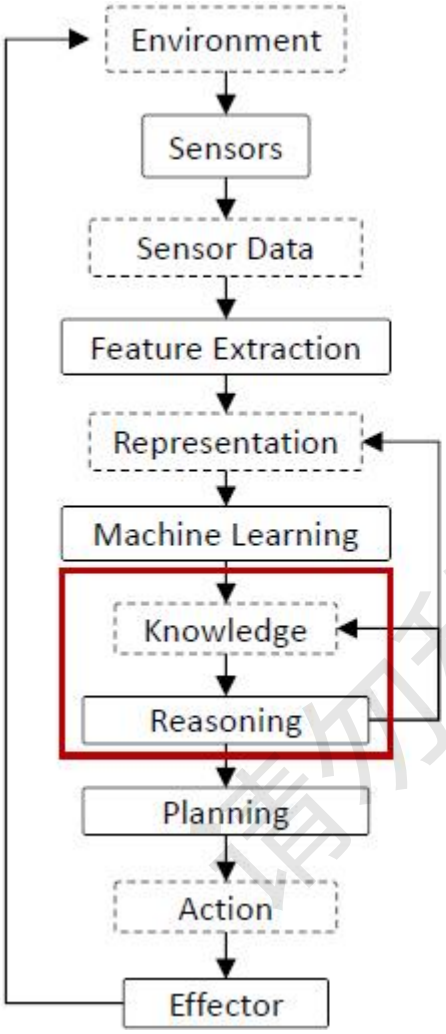


Image Recognition:
If it looks like a duck



Audio Recognition:
Quacks like a duck



学习结果：知识推理

Activity Recognition:
Swims like a duck

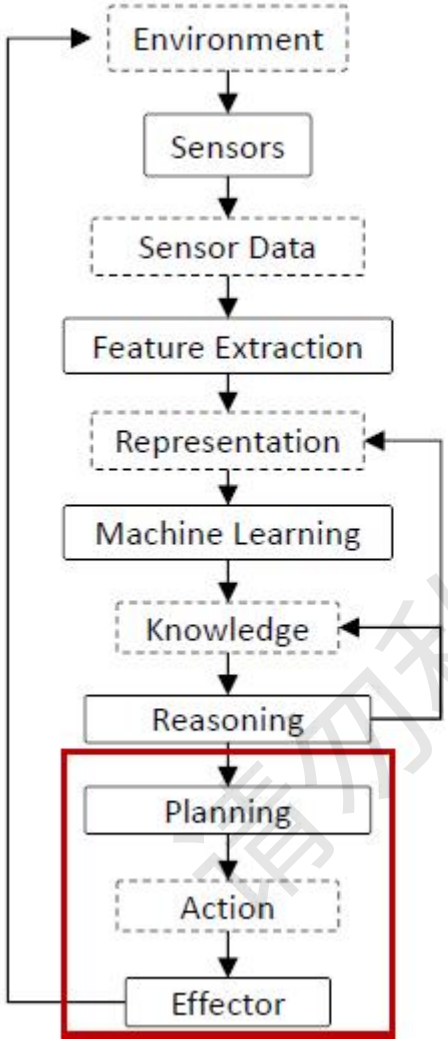


清华私自上传本课件

1.4 深度强化学习

- 从深度学习到深度强化学习

智能体



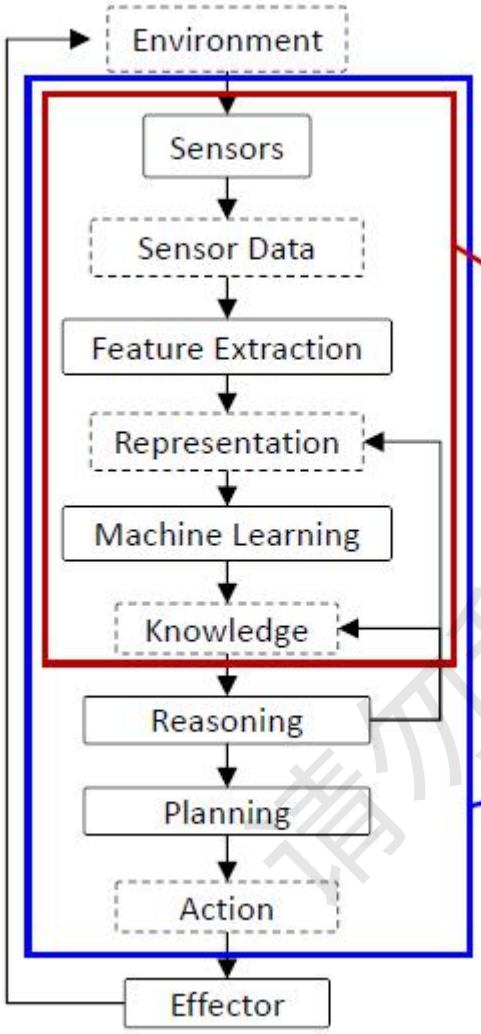


推理：行动规划》作用器



1.4 深度强化学习

- 从深度学习到深度强化学习



智能体

深度学习 → 场景感知



强化学习 → 行动决策

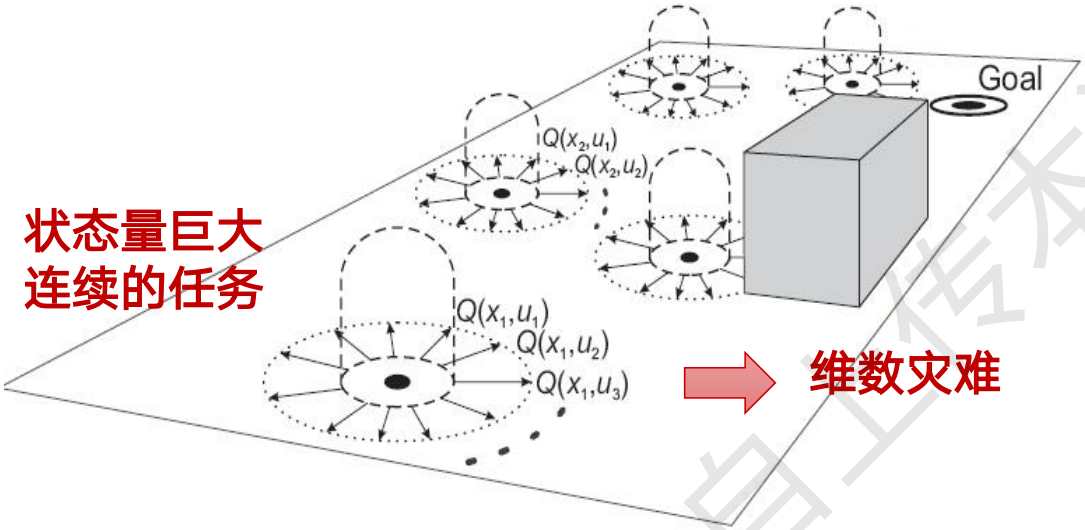
通用智能

Open AI解决方案

清华私自上传本课件

1.4 深度强化学习

■ 深度强化学习原理



状态量巨大
连续的任务

维数灾难

如何转化？

价值函数近似

$$Q(s,a,w) \approx Q^{\pi}(s,a)$$

神经网络　　动作价值函数

可以拟合任意复杂函数！

- ✓ 将强化学习和深度学习结合在一起：用强化学习来定义问题和优化目标，用深度学习来解决状态表示、策略表示等问题；
- ✓ 分别用深度神经网络来建模强化学习中的值函数、策略，然后用误差反向传播算法来优化目标函数；
- ✓ 深度强化学习在一定程度上是解决复杂问题的通用智能框架。

请勿私自上传本课件

1.4 深度强化学习

深度强化学习模型：Deep Q-Learning

核心设计

- 用深度神经网络作为Q值网络，参数为w

$$Q(s,a,w) \approx Q^{\pi}(s,a)$$

- 定义损失函数，均方差

$$L(w) = E[(r + \gamma \max_{a'} Q(s',a',w) - Q(s,a,w))^2]$$

Target　　Prediction

- 计算梯度，并更新参数

$$\frac{\partial L(w)}{\partial w} = E[(r + \gamma \max_{a'} Q(s',a',w) - Q(s,a,w)) \frac{\partial Q(s,a,w)}{\partial w}]$$

请勿私自上传本课件

1.4 深度强化学习

- **深度强化学习模型：Deep Q-Learning**

损失函数

$$L(w) = \mathbb{E}[(\underbrace{r + \gamma \max_{a'} Q(s', a', w)}_{Target} - Q(s, a, w))^2]$$

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$ 贪心策略
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$ 存储到经验池
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$从经验池采样
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

DQL算法伪码
NIPS2013

详细阅读：Mnih, Volodymyr, et al. "Playing atari with deep reinforcement learning." NIPS 2013.

请勿私自上传本课件

1.4 深度强化学习

■ 深度强化学习模型：DQN

算法 14.5: 带经验回放的深度Q网络

输入: 状态空间 $\mathcal{S}$，动作空间 $\mathcal{A}$，折扣率 γ，学习率 α

初始化经验池 $\mathcal{D}$，容量为 N;

随机初始化 Q 网络的参数 ϕ;

随机初始化目标 Q 网络的参数 $\hat{\phi} = \phi$;

repeat

  初始化起始状态 s;

  repeat

    在状态 s，选择动作 $a = \pi^{\epsilon}$;

    执行动作 a，观测环境，得到即时奖励 r 和新的状态 s'

    将 s, a, r, s' 放入 $\mathcal{D}$ 中;

    从 $\mathcal{D}$ 中采样 ss, aa, rr, ss';

$$y = \begin{cases} rr, & ss' \text{为终止状态}, \\ rr + \gamma \max_{a'} Q_{\hat{\phi}}(\mathrm{ss}', \mathrm{a}'), & \text{否则} \end{cases};$$

    以 $(y - Q_{\phi}(\mathrm{ss}, \mathrm{aa}))^2$ 为损失函数来训练 Q 网络;

    $s \leftarrow s'$;

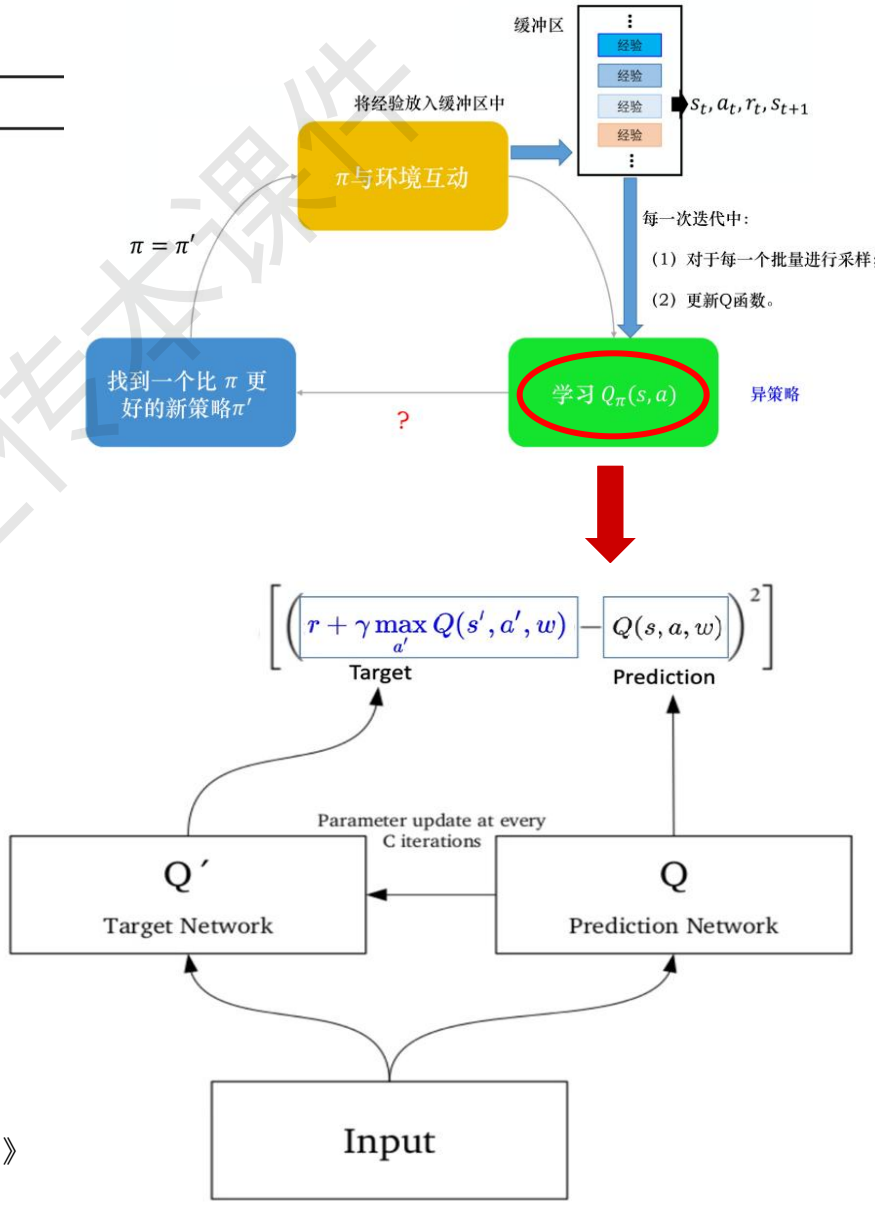
    每隔 C 步，$\hat{\phi} \leftarrow \phi$;

  until s 为终止状态;

until $\forall s, a, Q_{\phi}(\mathrm{s}, \mathrm{a})$ 收敛;

输出: Q 网络 $Q_{\phi}(\mathrm{s}, \mathrm{a})$

来源：《神经网络与深度学习》



请勿私自上传本课件

1.4 深度强化学习

■ 深度强化学习模型：DQN

网络	参数符号	是否参与反向传播?	更新频率	更新方式
Prediction Network（主网络）	θ	✅ 是	**每一步（或每个 batch）**	梯度下降：$\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$
Target Network（目标网络）	θ^-	❌ 否	**每隔 C 步（如 C=1000）**	硬复制：$\theta^- \leftarrow \theta$

关键点：

- 主网络频繁更新，用于预测当前 Q 值；
- 目标网络冻结一段时间，用于提供稳定的 TD 目标；
- 二者不是同步更新，而是“主网络跑，目标网络偶尔抄作业”。

如果没有目标网络（即 Q'=Q）：当你更新 θ 时，预测值和目标值同时变化 → 就像“追着自己的尾巴跑”，容易发散。

引入目标网络后：y 由固定参数 θ^- 决定，在 C 步内不变。所以优化目标是静态的，梯度方向更可靠。

请勿私自上传本课件

Agent与深度强化学习

1.1 什么是Agent

1.2 Agent结构设计

1.3 强化学习概述

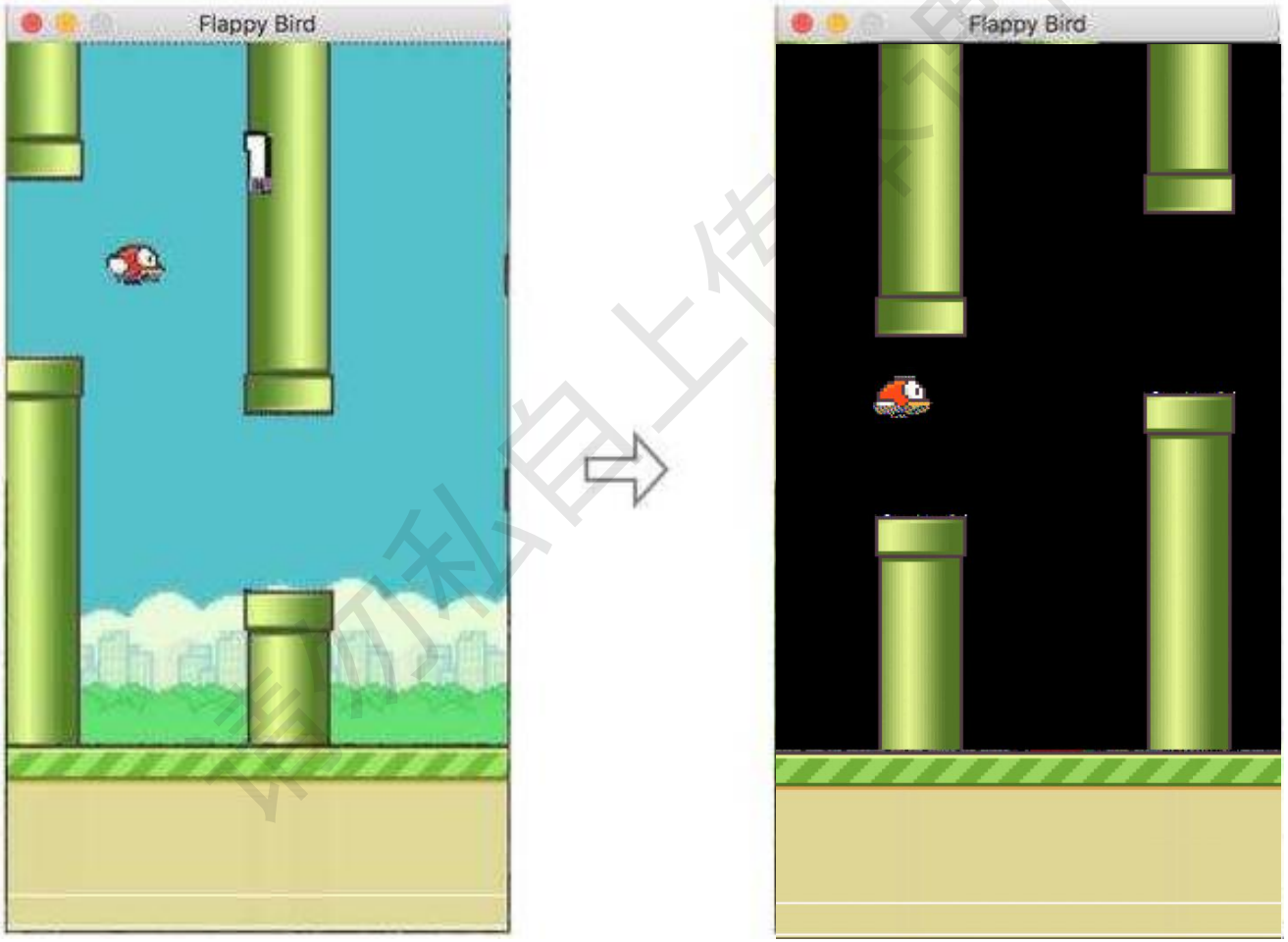
1.4 深度强化学习

1.5 应用案例分析

请勿私自上传本课件

1.5 应用案例分析

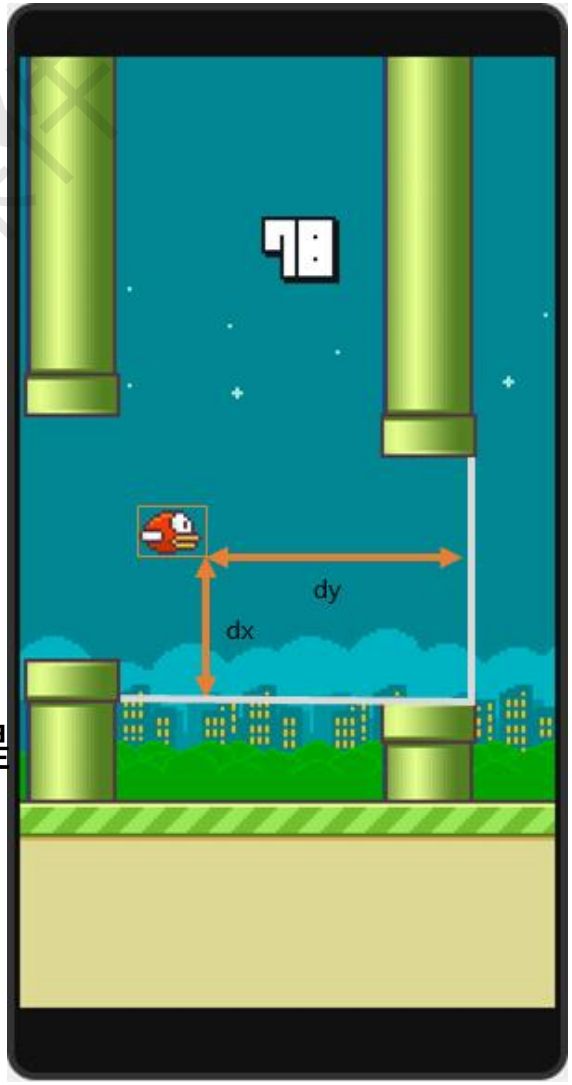
- Flappy Bird



1.5 应用案例分析

Flappy Bird

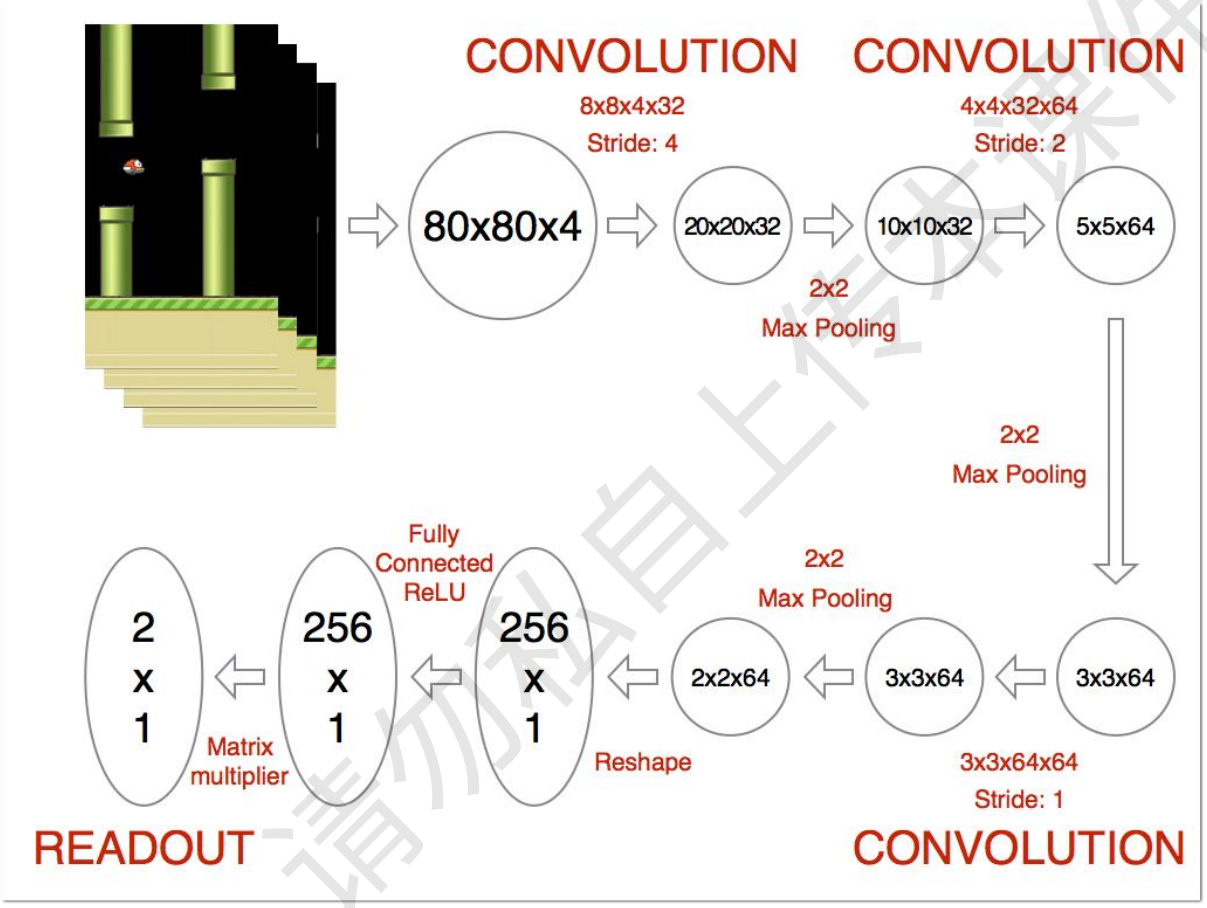
- ✓ Q-learning
  - 难点：识别小鸟、 柱子在图片中的位置
  - 解决方法：人工提取特征， 需要近千行代码
- ✓ Deep Q Network
  - 算法具备通用性
  - 采用End-to-End的训练方式， 无需人工提取Feature
  - 总共100多行代码



请勿私自上传课件

1.5 应用案例分析

- Flappy Bird实现



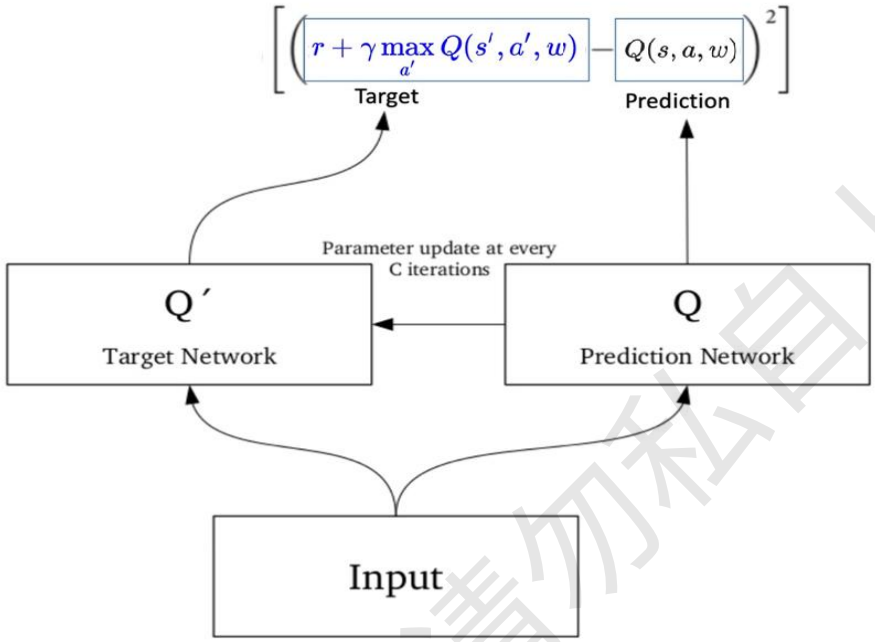
学习架构

深度学习CNN
+
强化学习DQN

```
git clone https://github.com/yenchenlin1994/DeepLearningFlappyBird.git
cd DeepLearningFlappyBird
python deep_q_network.py
```

1.5 应用案例分析

- Flappy Bird




```
# 初始化参数
initialize replay_memory D with max_size N
initialize Q-network with random weights θ
initialize target Q-network with weights θ' = θ

# 训练过程
for episode in range(max_episodes):
    state = env.reset() # 重置环境，获取初始状态
    done = False
    while not done:
        # 选择动作
        if random.random() < epsilon:贪心策略
            action = env.random_action() # 随机选择动作
        else:
            action = argmax(Q(state; θ)) # 选择Q值最大的动作
        # 执行动作，获取下一个状态和奖励
        next_state, reward, done = env.step(action)
        # 存储经验到回放记忆
        D.add(state, action, reward, next_state, done)
        # 从回放记忆中随机采样
        if len(D) > batch_size:
            batch = D.sample(batch_size)
            for s, a, r, s', terminal in batch:
                target = r + γ * max(Q(s'; θ') if not terminal else 0)
                # 更新Q网络
                Q(s, a; θ) ← Q(s, a; θ) + α * (target - Q(s, a; θ))
        # 更新状态
        state = next_state
    # 每隔一定步数更新目标网络
    if episode % target_update_frequency == 0:
        θ' = θ
```

请勿私自上传本课件

1.5 应用案例分析

- Flappy Bird实现

```python
class DQN(nn.Module):
    def __init__(self,c,action_dim,batch_size):
        super(DQN,self).__init__()
        self.batch_size = batch_size
        self.c = c
        self.conv1 = nn.Conv2d(c,64,(8,8),stride=4,padding=2)
        self.pooling1 = nn.MaxPool2d((2,2))
        self.conv2 = nn.Conv2d(64,128,(4,4),stride=2,padding=1)
        self.pooling2 = nn.MaxPool2d((2, 2),stride=1)
        self.conv3 = nn.Conv2d(128,128,(2,2),stride=2)
        self.fc1 = nn.Linear(128*4,128*6)
        self.fc2 = nn.Linear(128*6,32)
        self.fc3 = nn.Linear(32,action_dim)

    def forward(self, s):
        out = F.relu(self.conv1(s))
        out = self.pooling1(out)
        out = F.relu(self.conv2(out))
        out = self.pooling2(out)
        out = F.relu(self.conv3(out))
        out = out.view(-1,128*2*2)
        out = F.relu(self.fc1(out))
        out = F.relu(self.fc2(out))
        out = self.fc3(out)

        return out
```

```python
def train(self):
    if self.memory.counter < self.batch_size:
        return

    state_batch, action_batch, next_state_batch, reward_batch, done_batch = self.memory.sample()

    state = Variable(torch.from_numpy(state_batch))
    action = Variable(torch.from_numpy(action_batch)).float()
    next_state = Variable(torch.from_numpy(next_state_batch))
    reward = Variable(torch.from_numpy(reward_batch).view(self.batch_size,1))
    done = Variable(torch.from_numpy(done_batch).view(self.batch_size,1)).float()

    q_next = self.DQN(next_state).max(1)[0].view(self.batch_size,1)
    q_target = reward + self.gamma*done*q_next
    q_target.detach_()
    q_eval = torch.sum(self.DQN(state)*action,dim=1).unsqueeze(-1)

    loss = F.smooth_l1_loss(q_eval,q_target)
    self.optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm(self.DQN.parameters(), 0.8)
    self.optimizer.step()
```